sistemas inteligentes de computación

Regresión

Dr. Pedro L. Galindo Riaño
Universidad de Cádiz

www2.uca.es/grup-invest/sic
www2.uca.es/dept/leng_sist_informaticos/preal/pedro

Regression Analysis…

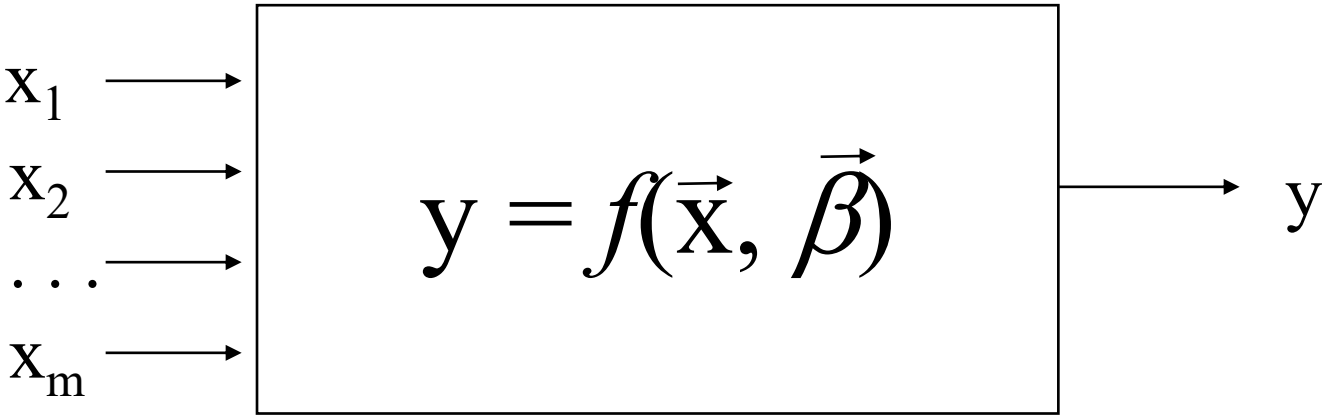
- *Our objective is to **analyze the relationship** between interval variables;*
- *Regression analysis is used to predict the value of one variable (the **dependent variable**) on the basis of other variables (the **independent variables**).*
- *Dependent variable: denoted **Y***
- *Independent variables: denoted* $X_1, X_2, \ldots, X_m$

Componentes de un modelo de regresión

- *Los modelos de regresión incluyen los siguientes componentes:*
  - Los parámetros desconocidos (vector) $\vec{\beta}$
  - Las variables independientes (vector) $\vec{x}_i$
  - La variable dependiente (escalar) y_i
  - El error cometido (escalar) e_i

$$y_i = f(\vec{x}_i, \vec{\beta}) + e_i \qquad i=1,2,..N$$

Modelo de regresión generico



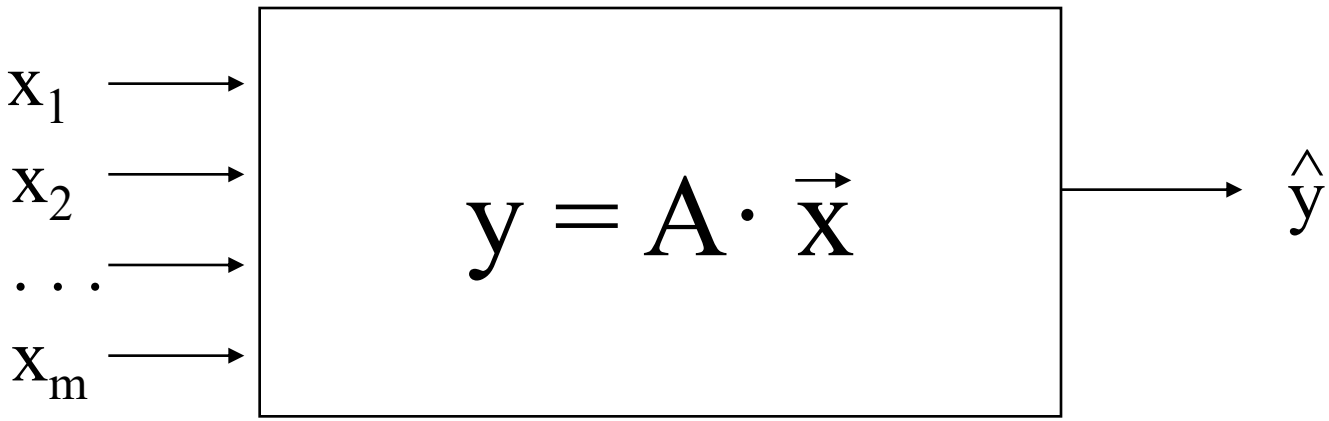
x : entradas al sistema

β : parámetros internos del sistema

y : salida del sistema

Modelo de regresión Lineal

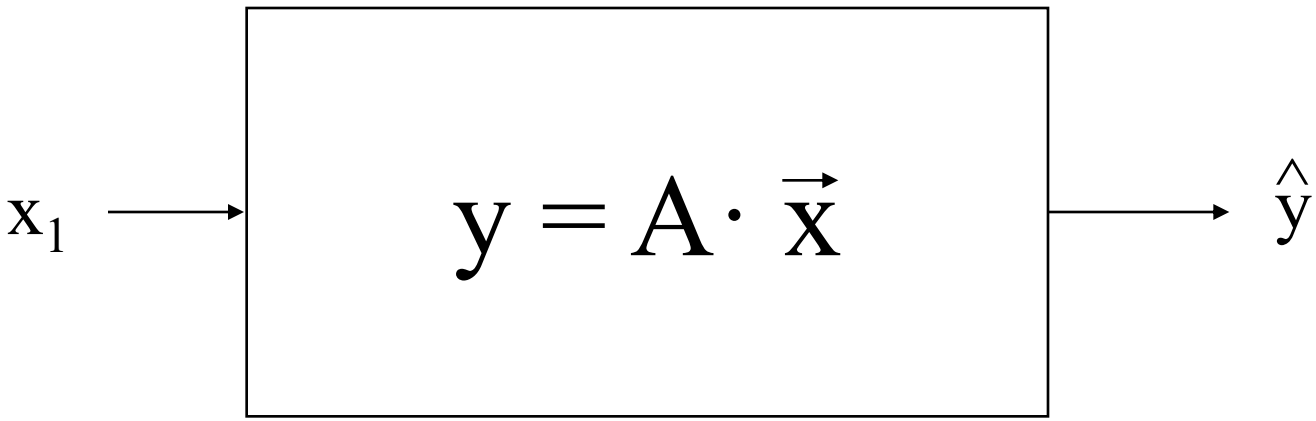
Modelo Lineal Genérico



$$\hat{y} = a_1 x_1 + a_2 x_2 + \ldots + a_m x_m + a_0$$

Modelo Lineal
(1 entrada)

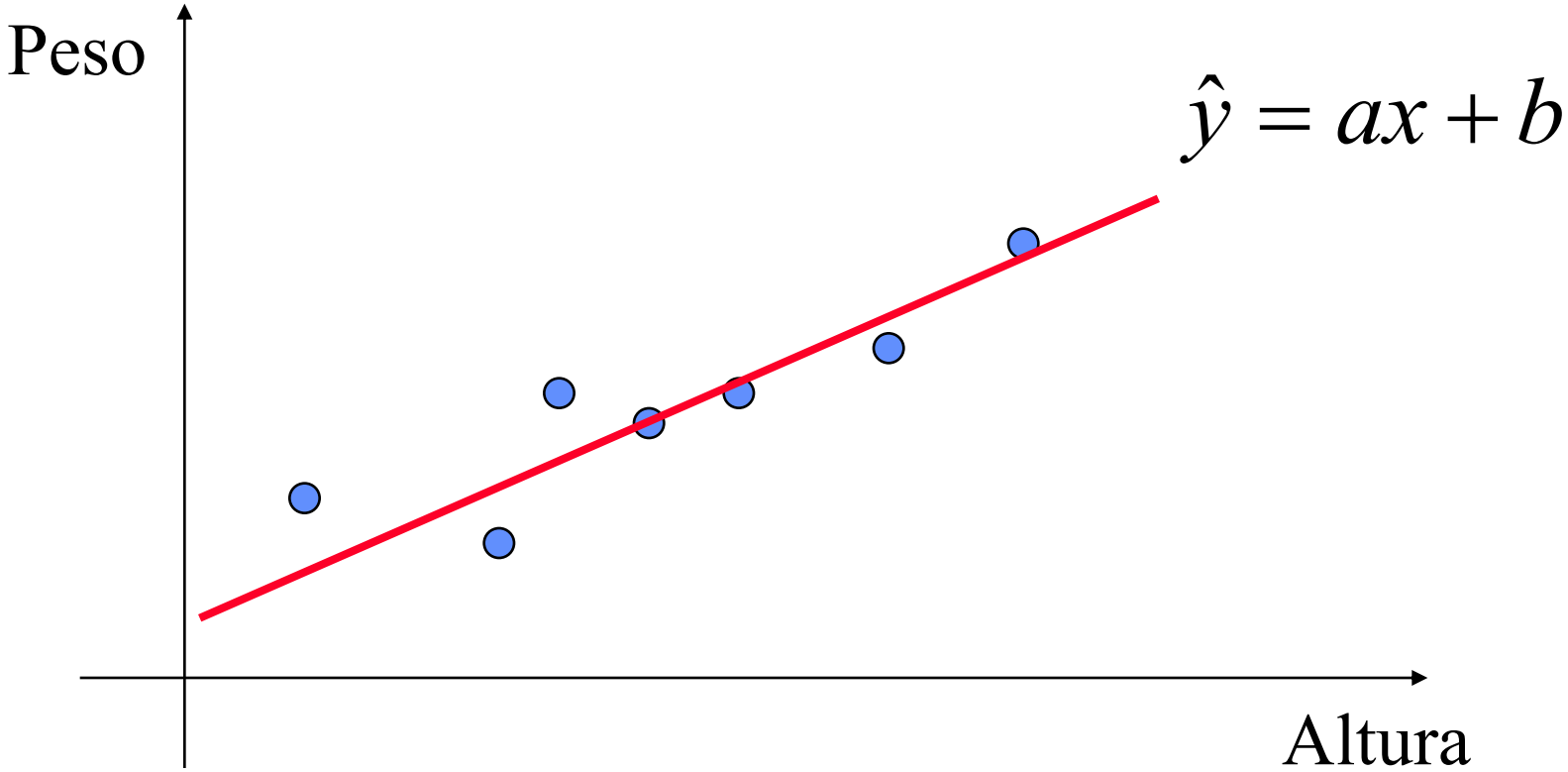
Modelo Lineal(dimensión=1)



$$\hat{y} = a_1 x_1 + a_0$$

Modelo lineal

Ajuste a una recta (m=1)



Modelo lineal (m=1)
Solución Ordinary Least Squares (OLS)

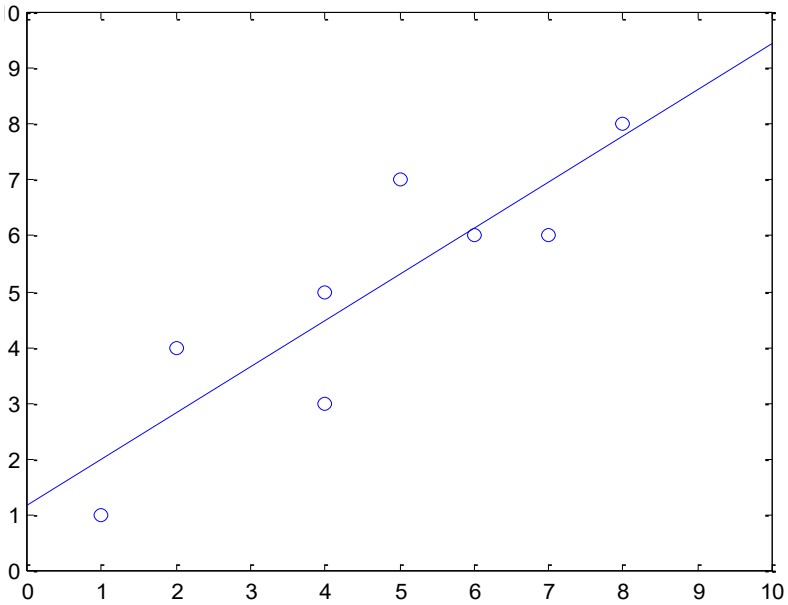
$$\begin{pmatrix} \sum_{i=1}^{n} x_i^{\,2} & \sum_{i=1}^{n} x_i \\ \sum_{i=1}^{n} x_i & n \end{pmatrix} \underbrace{\begin{pmatrix} a \\ b \end{pmatrix}}_{\vec{\beta}} = \begin{pmatrix} \sum_{i=1}^{n} y_i x_i \\ \sum_{i=1}^{n} y_i \end{pmatrix}$$

(son las incógnitas del sistema)

Modelo lineal

Ajuste a una recta (m=1)

```
x=[1 2 4 4 5 6 7 8];
y=[1 4 3 5 7 6 6 8];

A=[sum(x.*x)    sum(x)
     sum(x)     length(x)];
b=[sum(x.*y) sum(y)]';
sol=A\b;

plot (x,y,'o'); hold on;
axis([0 10 0 10]);
xr=[0 10];
yr=[sol(2) sol(1)*10+sol(2)];
plot (xr,yr,'r'); hold off;
error = y - ( sol(1)*x + sol(2) );
E = sum(error.^2)
» Sol =
    0.8276
    1.1724
» E = 8.6897
```



Residual

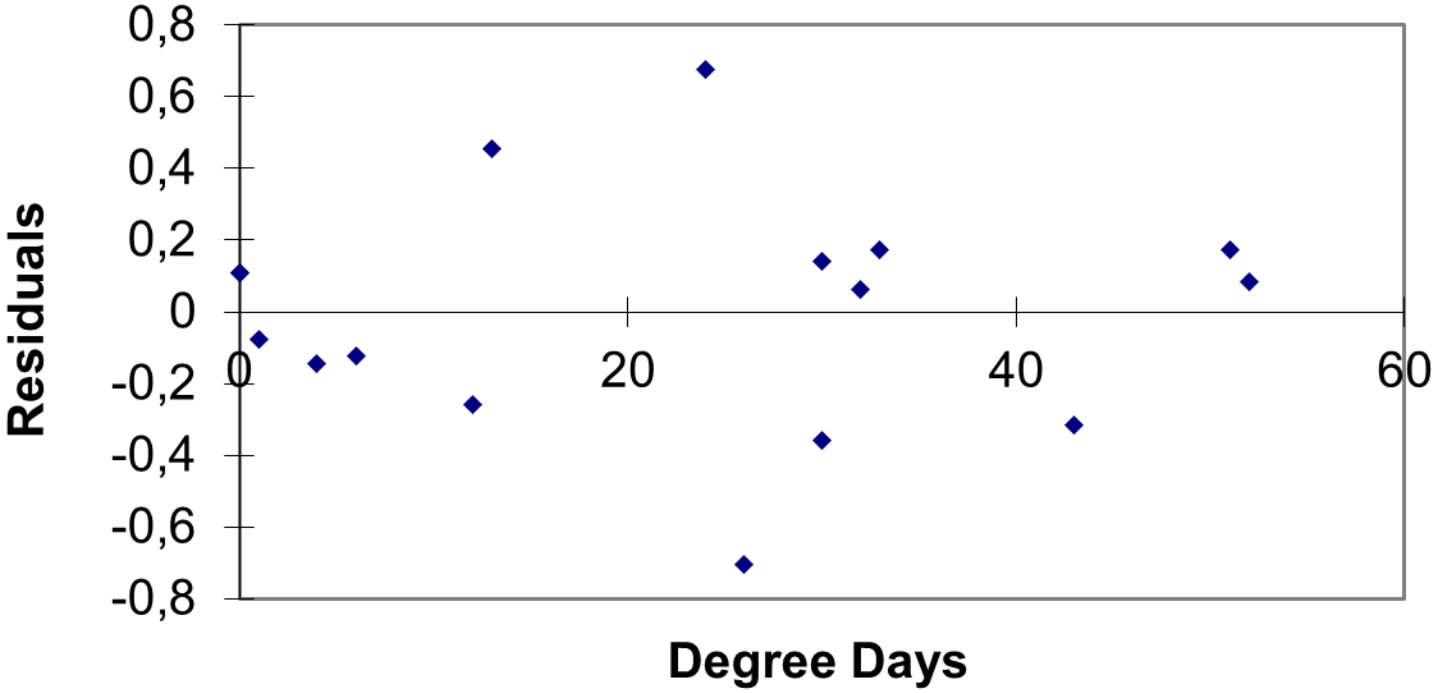
- *The difference between the observed value y_i and the corresponding fitted value $\hat{y}_i$*

$$e_i = y_i - \hat{y}_i$$

- *Residuals are highly useful for studying whether a given regression model is appropriate for the data at hand.*

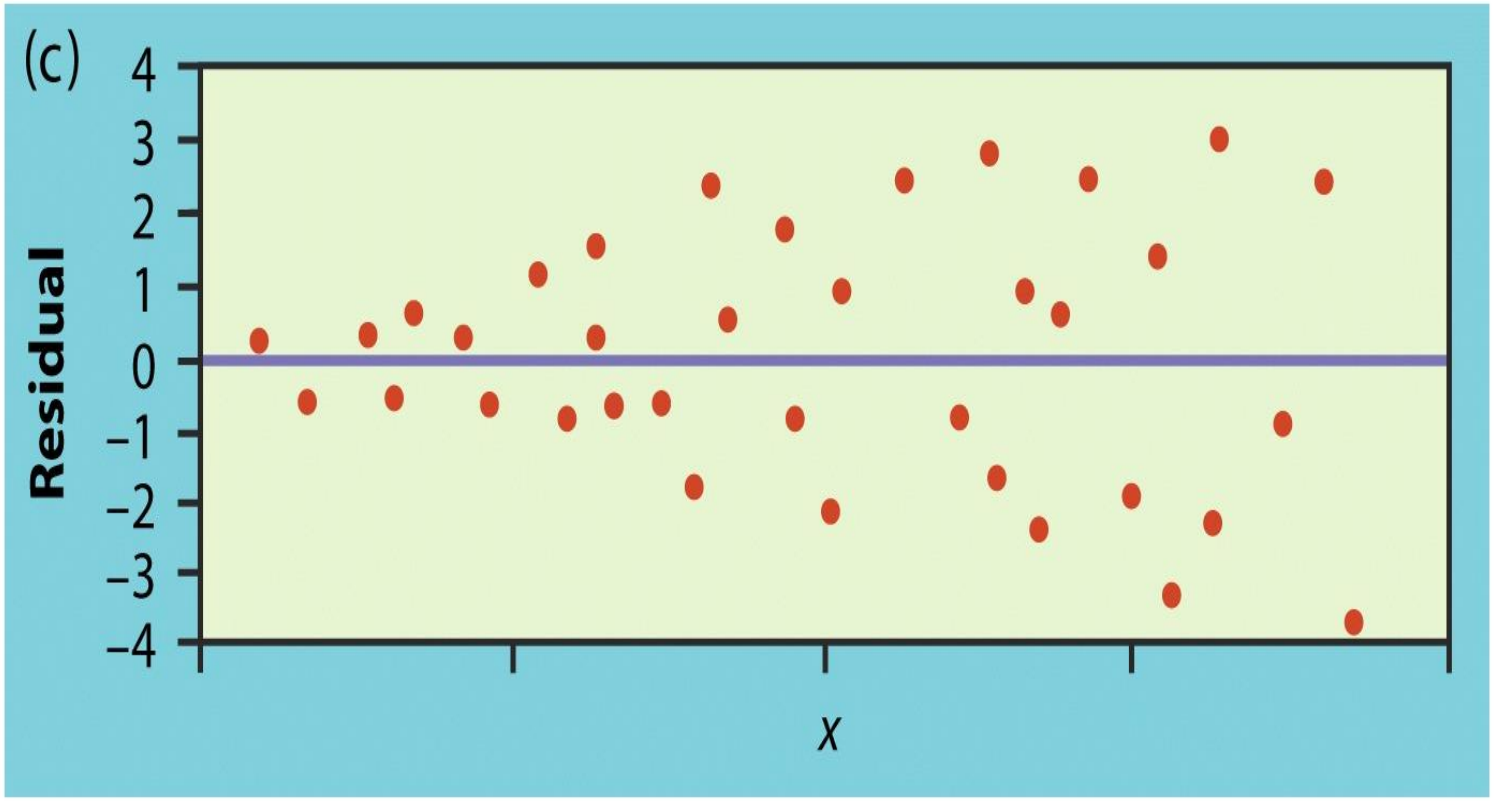
Residual plots

- *The residuals should have no systematic pattern*



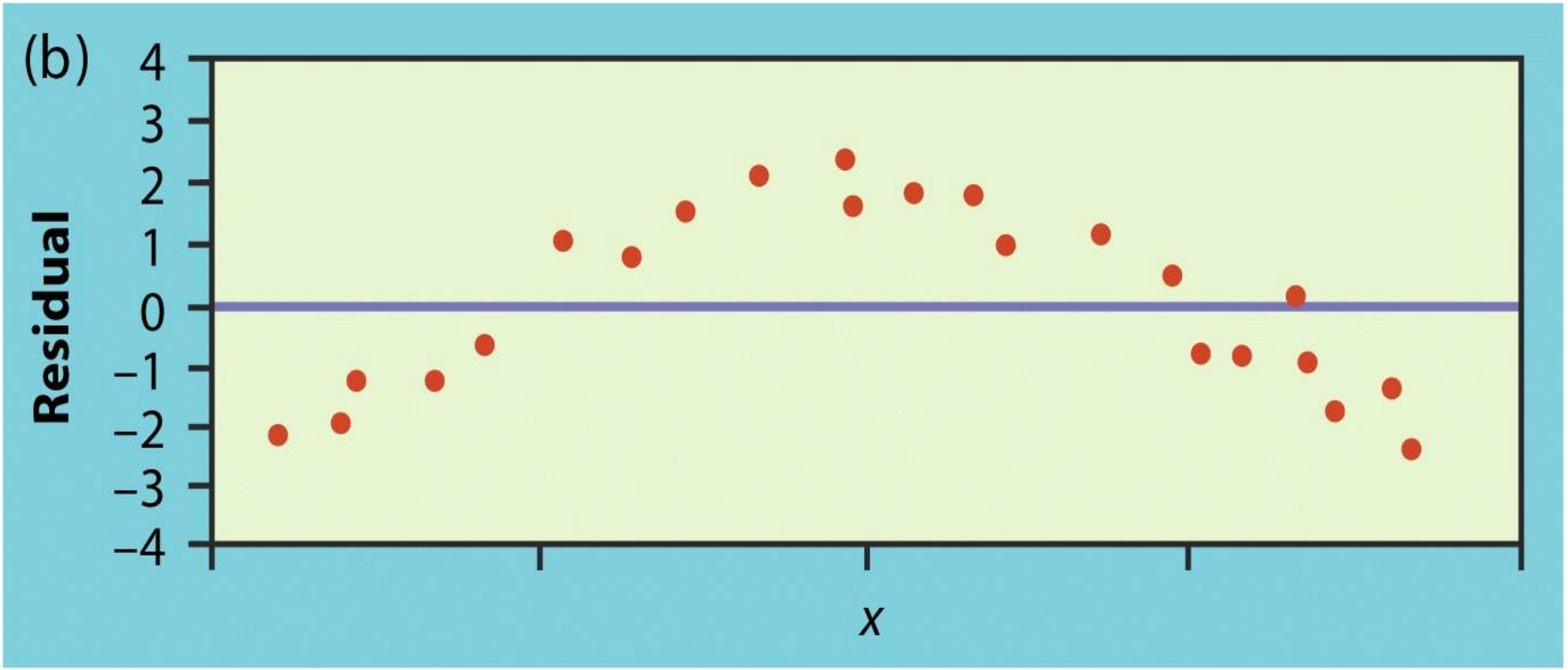
Residual plots

- *The points in this plot show more spread for larger values of the explanatory variable x, so prediction will be less accurate when x is large.*



Residual plots

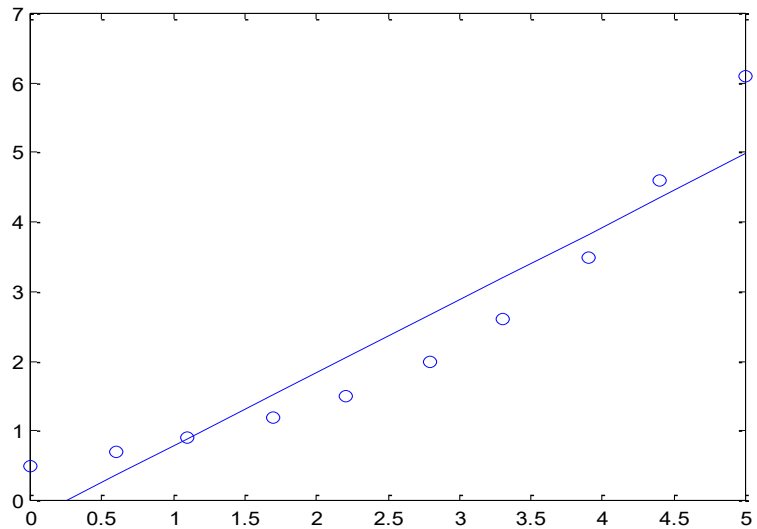
- *The points in this residual plot have a curve pattern, so a straight line fits poorly*

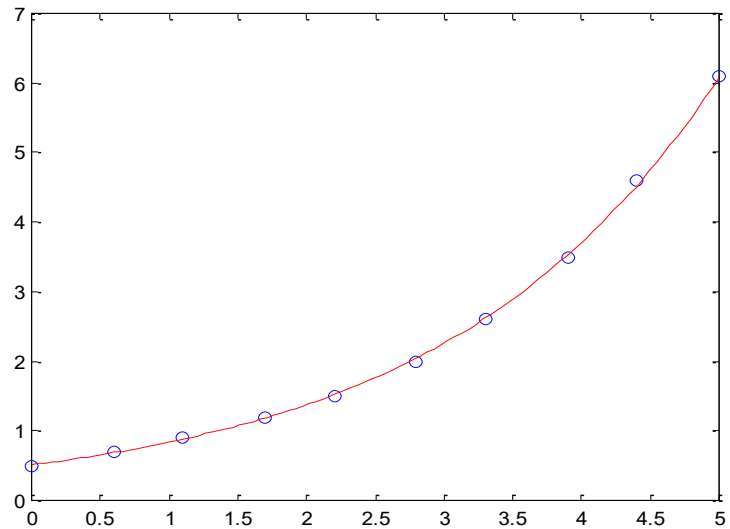


Reducción modelos no lineales al OLS (1 sola entrada)

Reducción a un modelo lineal

Modelo exponencial





- *Cuando el modelo lineal es insuficiente, es posible reducir algunos modelos al modelo lineal mediante un cambio de variables*

Variable Transformations

- *If the residual plot suggests that the variance is not constant, a transformation can be used to stabilize the variance.*
- *If the residual plot suggests a non linear relationship between x and y, a transformation may reduce it to one that is approximately linear.*
- *Common linearizing transformations are:*

$$\frac{1}{x}, \quad \log(x)$$

- *Variance stabilizing transformations are:*

$$\frac{1}{y}, \quad \log(y), \quad \sqrt{y}, \quad y^2$$

Reducción a un modelo lineal

Modelo exponencial

- *Sea la curva:*

$$y = Ae^{Bx}$$

- *Tomando logaritmos:*

$$\ln(y) = Bx + \ln(A)$$

- *Y hacemos el cambio de variables:*

$$\hat{y} = \ln(y) \qquad C = \ln(A)$$

$$\hat{y} = Bx + C$$

Reducción a un modelo lineal

Modelo exponencial

```
x=[0 0.6 1.1 1.7 2.2 2.8 3.3 3.9 4.4 5]';
y=[0.5 0.7 0.9 1.2 1.5 2 2.6 3.5 4.6 6.1]';

% cambio de variable en los datos
yp=log(y);

% solucion del problema lineal
sol = polyfit(x,yp,1);
B=sol(1); C=sol(2);

% calculamos los parámetros de la exponencial
A=exp(C);
plot (x,y,'o');
axis([0 5 0 7]); hold on;
ex = 0:0.1:5;
plot (ex,A*exp(B*ex),'r'); hold off;
```

Reducción a un modelo lineal
Otros modelos

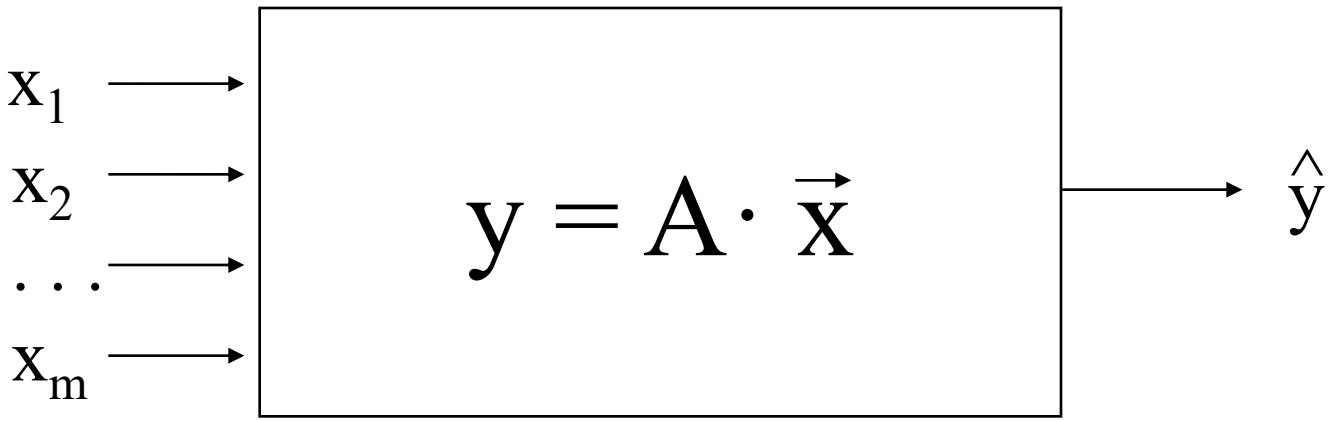
- *De forma similar, se pueden reducir los modelos siguientes al general:* $\hat{y} = A \cdot \hat{x} + B$

Función	Modelo	Cambio de vars.
$y = C \cdot x^A$	$\ln(y) = A \cdot \ln(x) + \ln(C)$	$\hat{y} = \ln(y)$ $\hat{x} = \ln(x)$ $C = e^B$
$y = C \cdot x \cdot e^{Ax}$	$\ln\left(\frac{y}{x}\right) = A \cdot x + \ln(C)$	$\hat{y} = \ln\left(\frac{y}{x}\right)$ $C = e^B$

Solución modelo lineal con varias entradas

Generalized Linear Model (GLM)

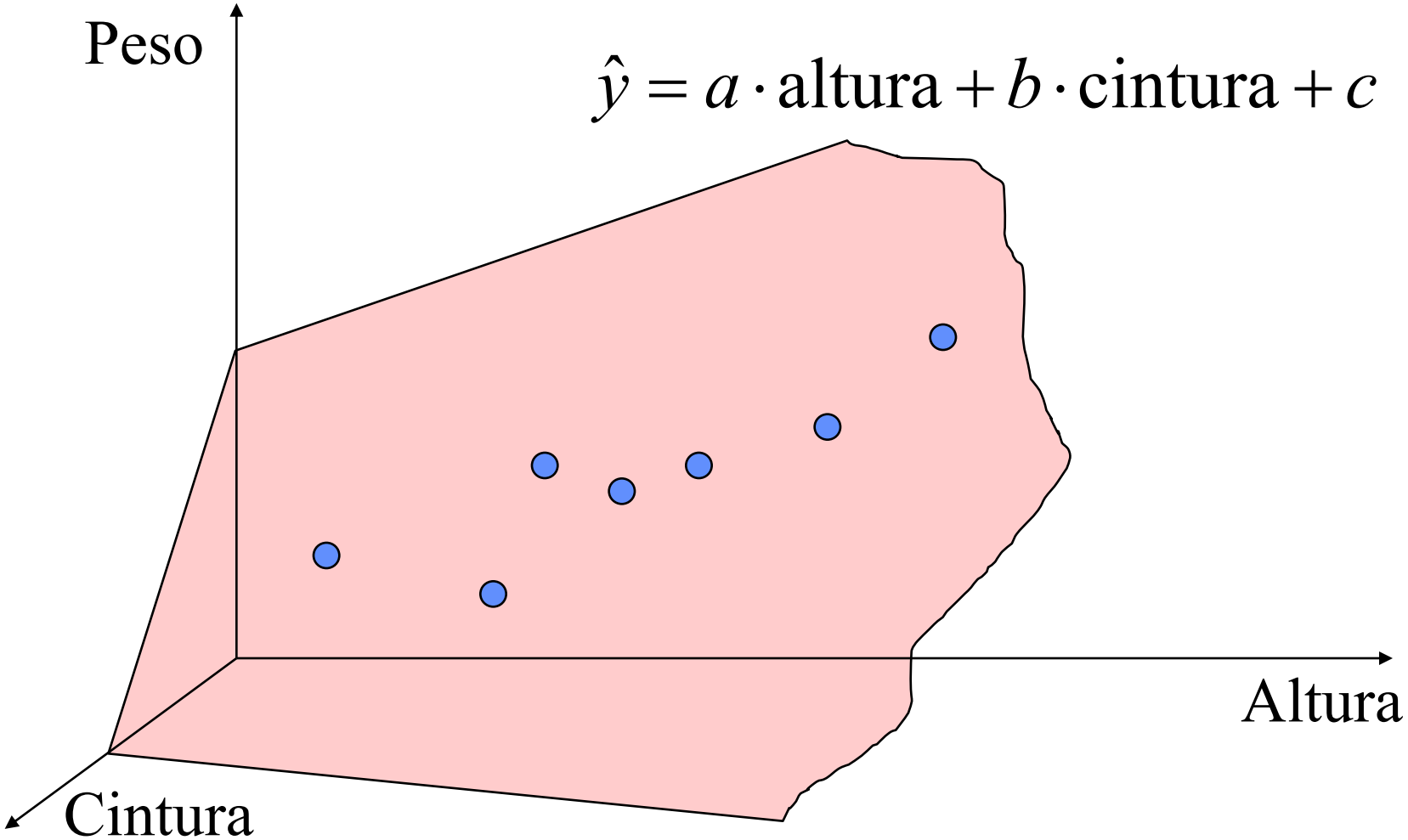
Modelo lineal generalizado (GLM) (m>1)



$$\hat{y} = a_1 x_1 + a_2 x_2 + \ldots + a_m x_m + a_0$$

Modelo lineal

Ajuste a un plano (m=2)



¿Cómo calculamos el hiperplano que mejor se ajusta a los datos?

Solución GLM (Construcción de A)

- *Luego A se construiría de la siguiente forma:*

$$A_{NxM} = \begin{bmatrix} \uparrow & \uparrow & \uparrow & \uparrow & \uparrow & \uparrow \\ x_1 & x_2 & x_3 & \dots & x_m & 1 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \end{bmatrix}$$

N = numero de datos

M = número de características + 1

Solución GLM

$$\boxed{coefs = \underbrace{\left(A^t \cdot A\right)^{-1} \cdot A^t}_{\text{Pseudoinversa de A}} \cdot y}$$

Modelo lineal (m=1)
Expresión matricial

$$\boxed{a_1 \cdot x + a_0 = y}$$

x	y
2	4.2
4	7.6
3	5.6
5	9.8
2	4.1
6	12.5
1	0.9

$$\begin{aligned}
a_1 \cdot 2 + a_0 &= 4.2 \\
a_1 \cdot 4 + a_0 &= 7.6 \\
a_1 \cdot 3 + a_0 &= 5.6 \\
a_1 \cdot 5 + a_0 &= 9.8 \\
a_1 \cdot 2 + a_0 &= 4.1 \\
a_1 \cdot 6 + a_0 &= 12.5 \\
a_1 \cdot 1 + a_0 &= 0.9
\end{aligned}$$

$$\cdot \begin{bmatrix} 2 & 1 \\ 4 & 1 \\ 3 & 1 \\ 5 & 1 \\ 2 & 1 \\ 6 & 1 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} a_1 \\ a_0 \end{bmatrix} = \begin{bmatrix} 4.2 \\ 7.6 \\ 5.6 \\ 9.8 \\ 4.1 \\ 12.5 \\ 0.9 \end{bmatrix}$$

$$A_{7x2} * \ coefs_{2x1} = y_{7x1}$$

Modelo lineal (m=1)
Solución matricial

$$A_{7x2} \cdot \text{coefs}_{2x1} = y_{7x1}$$

$$A'_{2x7} \cdot A_{7x2} \cdot \text{coefs}_{2x1} = A'_{2x7} \cdot y_{7x1}$$

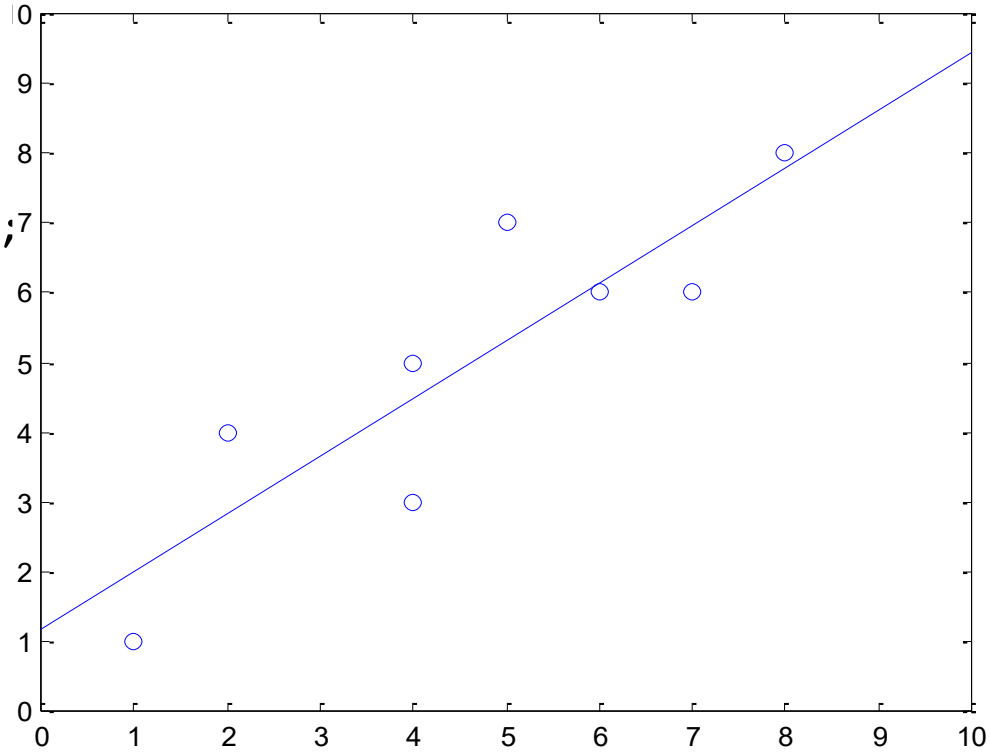
$$(A'_{2x7} \cdot A_{7x2})^{-1} \cdot (A'_{2x7} \cdot A_{7x2}) \cdot \text{coefs}_{2x1} = (A'_{2x7} \cdot A_{7x2})^{-1} A'_{2x7} \cdot y_{7x1}$$

$$\text{coefs}_{2x1} = (A'_{2x7} \cdot A_{7x2})^{-1} A'_{2x7} \cdot y_{7x1}$$

Modelo lineal = GLM, con n=1

Ej: Ajuste a una recta

```
x=[1 2 4 4 5 6 7 8]';
y=[1 4 3 5 7 6 6 8]';
A = [x ones(size(x))];
sol = inv(A'*A)*(A'*y);
r = y-A*sol;
E = r'*r;
» Sol =
    0.8276
    1.1724
» E = 8.6897
```

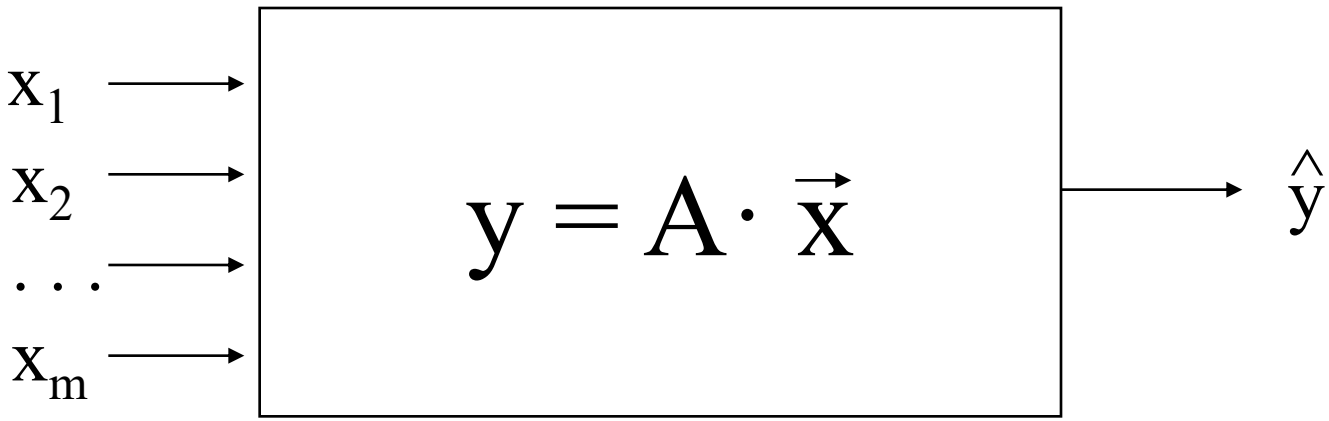


Visualización de la superficie de error E(a,b)

```
x=[1 2 4 4 5 6 7 8]'-4;
y=[1 4 3 5 7 6 6 8]';
A = [x ones(size(x))];

% Solucion exacta
sol=pinv(A)*y

% Valores de a y b a evaluar
I = -5:0.01:10;
[a,b]=meshgrid(I,I);
E=zeros(size(a));

% Calculo del error para cada
% valor de a y b
for i=1:length(a(:))
  r = y-A*[a(i);b(i)];
  E(i)=r'*r;
end

% Posicion del minimo
[f,c]=find(E==min(E(:)));
a_estim = I(c)
b_estim = I(f)
```

```
contour(I,I,E);hold on;
plot(sol(1),sol(2),'*');
plot(a_estim,b_estim,'o');
hold off, legend('Error', 'exacta','estim')
xlabel('a','FontSize',15)
ylabel('b','FontSize',15)
```

Modelo Lineal Genérico



$$\hat{y} = a_1 x_1 + a_2 x_2 + \ldots + a_m x_m + a_0$$

GLM – modelo polinomial (grado 4)

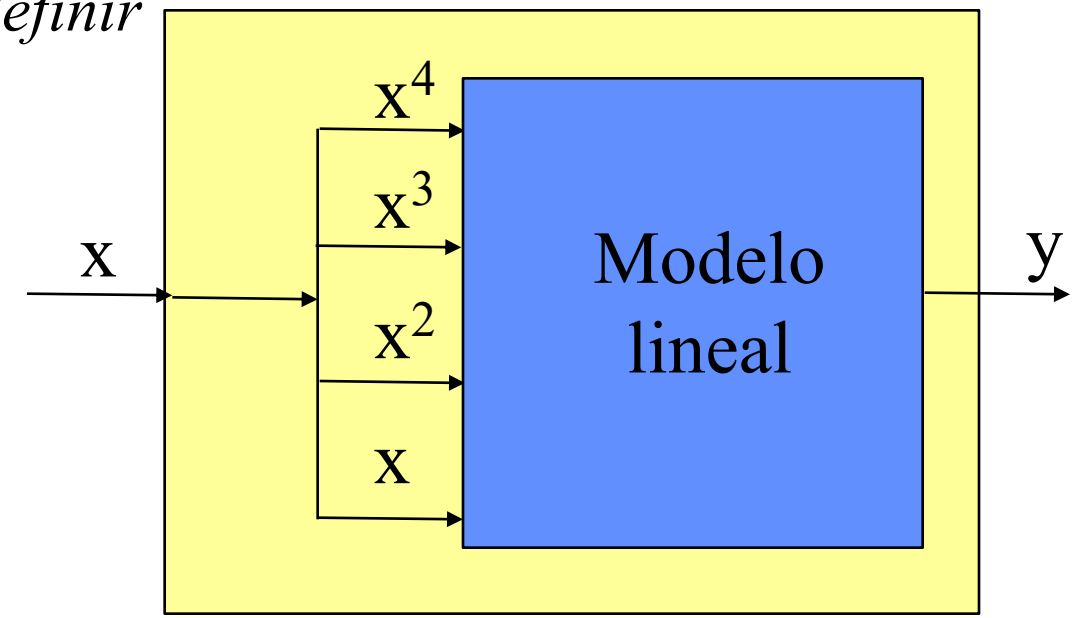
- *Un polinomio de orden 4 tiene la forma:*

  $$y=ax^4+bx^3+cx^2+dx+e$$

- *Para convertirlo en un modelo lineal:*

  $$y = ax_1+bx_2+cx_3+dx_4+e$$

- *Basta definir*



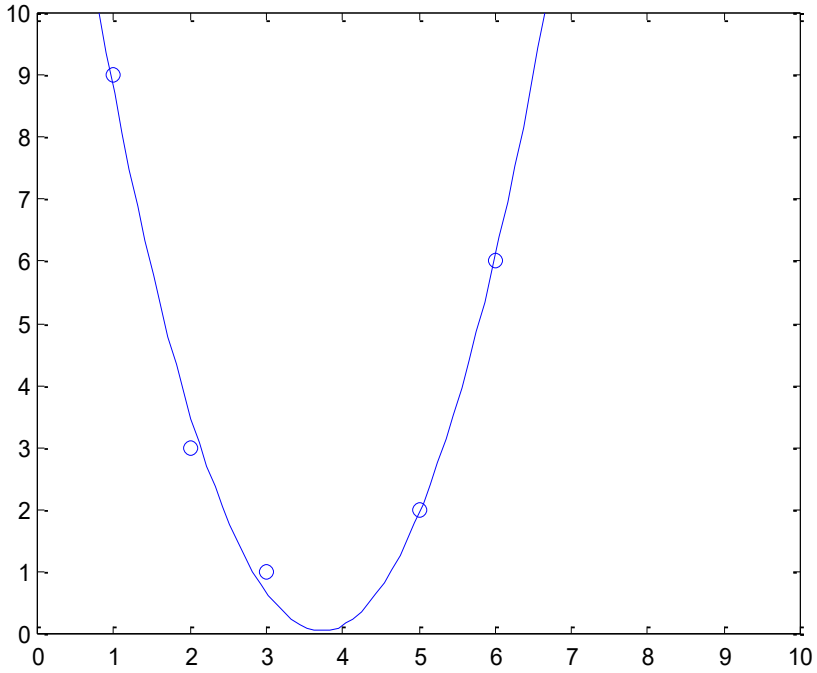
GLM – modelo polinomial (grado 4)

- *Luego A se construiría de la siguiente forma:*

$$A = \begin{bmatrix} \uparrow & \uparrow & \uparrow & \uparrow & \uparrow \\ x^4 & x^3 & x^2 & x & 1 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \end{bmatrix}$$

Ajuste polinómico (grado 2)
Solución usando GLM

```
x=[1 2 3 5 6]';
y=[9 3 1 2 6]';

A = [x.*x x ones(size(x))];
coefs = pinv(A)*y;

plot (x,y,'o');hold on;
axis([0 10 0 10]);
xi=linspace(0,10,100);
plot (xi,polyval(coefs,xi));
hold off;
```

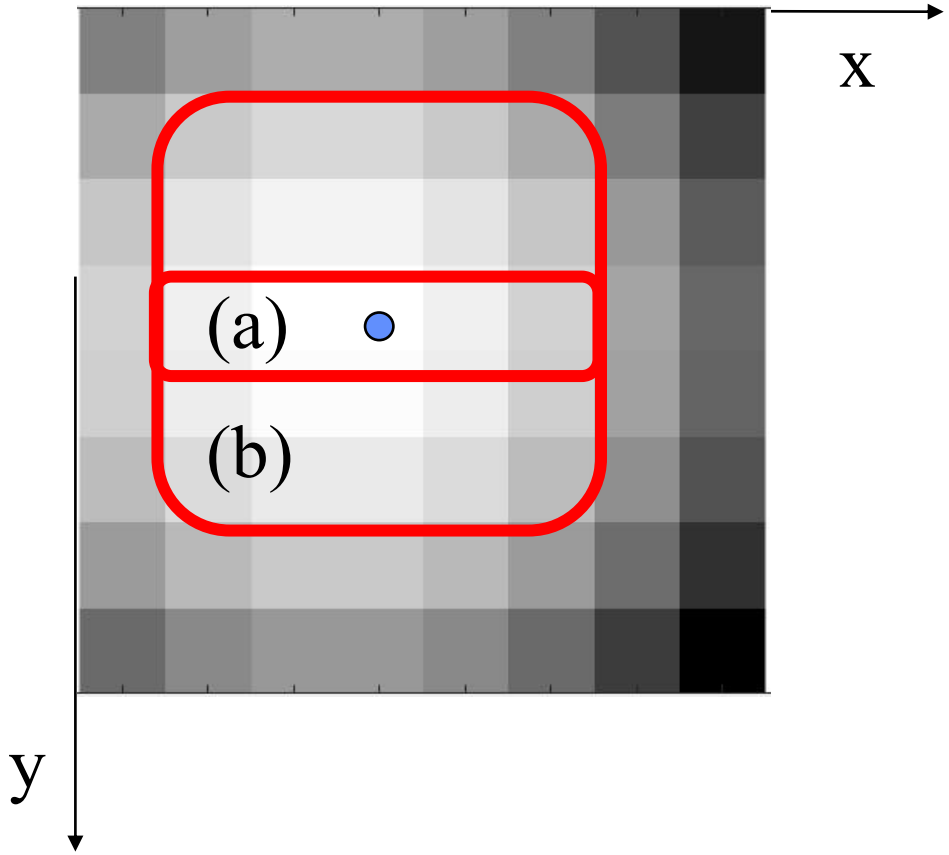


GLM – orden de las columnas

- *El orden de las columnas es indiferente*
- *Basta saber que los coeficientes resultantes al aplicar* `coefs=pinv(A)*y` *van asociados a cada columna*
- *Ej: si en el modelo polinomial de grado 2, definimos* `A=[ones(N,1) x x²]` *lo que ocurre es que:*
  - el primer coeficiente que se obtiene es el término independiente
  - el segundo es el coeficiente de x
  - el tercero el coeficiente de x^2

GLM – Dos características

- Calcular el máximo de una imagen 2D con resolución subpixel
- A) Resolver el problema ajustando una **parábola** a un entorno de pixels de la **fila** que contiene el máximo
- B) Una vez entendido, generalizarlo a 2D, ajustando una función **cuadrática bivariada** $ax^2+by^2+cxy+dx+ey+f$



GLM – Dos características

```
[x,y]=meshgrid(1:8,1:8);
% Generar imagen de prueba
A=(x-3.5).^2+(y-4.3).^2;
A=-A;

% Encontrar el máximo. ¿Cómo?
```

bivariate quadratic function

$$I(x,y)=ax^2+by^2+cxy+dx+ey+f$$

$$dI/dx = 2ax+cy+d=0$$

$$dI/dy = 2by+cx+e=0$$

$$\begin{vmatrix} 2a & c \\ c & 2b \end{vmatrix} \begin{vmatrix} x \\ y \end{vmatrix} = \begin{vmatrix} -d \\ -e \end{vmatrix}$$

```
% Generar imagen de prueba con un máximo en una posicion dada
[x,y]=meshgrid(1:8,1:8);
s=(x-3.517513).^2+(y-4.321653).^2;
s=-s;
subplot(2,1,2),imagesc(s), title('Datos')

% Encontrar el máximo de s con resolucion subpixel. ¿Cómo?

% A) Ajustar modelo lineal con una cónica
x=x(:); y=y(:); s=s(:);
A = [x.^2 y.^2 x.*y x y ones(size(y))];
coefs = pinv(A)*s;

% Podemos ver cuánto de bueno es nuestro modelo
s_estimada = reshape(A*coefs,8,8);
subplot(2,1,2),imagesc(s_estimada);title('Imagen ajustada')


% Una vez calculados los coeficientes, calcular el máximo de la función
ajustada
M = [2*coefs(1)  coefs(3)
     coefs(3)    2*coefs(2)];
t = [-coefs(4)
     -coefs(5)];

solucion = inv(M)*t
```

Fourier transform – señal = suma senos y cosenos

- *La idea es construir A de la siguiente forma:*

$$A = \begin{bmatrix} \uparrow & \uparrow & \uparrow & \uparrow & \uparrow & \uparrow & \uparrow \\ \cos(x) & \sin(x) & \cos(2x) & \sin(2x) & \cdots & \cos(nx) & \sin(nx) \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \end{bmatrix}$$

Cálculo fft usando GLM

```
function coefs = mifft(v)
  A = [];
  I = linspace(0, 2*pi, length(v) + 1)';
  I = I(1:end-1);
  for i = 0:length(v)-1
    A = [A cos(i*I) sin(i*I)];
  end
  coefs = pinv(A) * v(:);
  coefs = length(v) * complex(coefs(1:2:end),coefs(2:2:end))';
end
```

Métricas en regresión

Métricas de regresión

- *Mean squared error*

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- *Root mean squared error*

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}$$

- *Mean absolute error*

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- *Coeficiente de determinación (R^2)*

  ($0 \leq R^2 \leq 1$)

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

Coeficiente de determinación

```
% Datos
x = [1 2 3 4 5]';   y = [2 4 5 4 5]';

% Ajuste lineal (regresión de grado 1)
p = polyfit(x,y,1);
y_hat = polyval(p,x);
y_mean = mean(y);                  % Media de los datos

% Cálculo de sumas de cuadrados
SST = sum((y - y_mean).^2)         % Total
SSE = sum((y - y_hat).^2)          % Error (no explicado)
SSR = sum((y_hat - y_mean).^2)     % Explicado
R2 = 1 - SSE/SST                   % Coeficiente de determinación

% Comprobación de identidad fundamental
fprintf('Comprobación SST ≈ SSR + SSE: %.4f\n\n', SSR+SSE);

% Gráfica
figure;
plot(x,y,'o','LineWidth',1.5); hold on
plot(x,y_hat,'r','LineWidth',1.5)
yline(y_mean,'--k','Media')
legend('Datos','Ajuste lineal','Media','Location','best')
title(['Regresión lineal   R^2 = ', num2str(R2)])
xlabel('x')
ylabel('y')
```

Pearson's correlation coefficient

- *Para evaluar la relación entre las predicciones y los valores reales, es frecuente utilizar el coeficiente de correlación de Pearson*
- *Da un valor entre -1 y 1*

$$r = \frac{\sum (y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum (y_i - \bar{y})^2} \cdot \sqrt{\sum (\hat{y}_i - \bar{\hat{y}})^2}}$$

($-1 \leq r \leq 1$)

Coeficiente de correlacion de Pearson

```
% Datos
x = [1 2 3 4 5]';      y = [2 4 5 4 5]';

% Medias
x_mean = mean(x);      y_mean = mean(y);

% Centrado de variables
x_c = x - x_mean;      y_c = y - y_mean;

% Cálculo manual de r (definición matemática)
r_manual = sum(x_c .* y_c) / sqrt(sum(x_c.^2) * sum(y_c.^2));

% Cálculo usando función de MATLAB
r_matrix = corrcoef(x,y);
r_matlab = r_matrix(1,2);

% Interpretación básica
if abs(r_manual) < 0.3
    disp('Correlación débil')
elseif abs(r_manual) < 0.7
    disp('Correlación moderada')
else
    disp('Correlación fuerte')
end
```

Modelos de regresión NO lineales

Logistic Regression

Why use Logistic Regression?

- *The dependent variable in many problems is "limited" to a couple of categories (true/false, or 0/1, or yes/no)*

- *Example:*
  - Whether a patient is diabetic given her symptoms

Objectives of Logistic Regression

- In contrast to (e.g.) OLS, logistic regression is used when the dependent variable has only two possible categories, i.e., is binary.

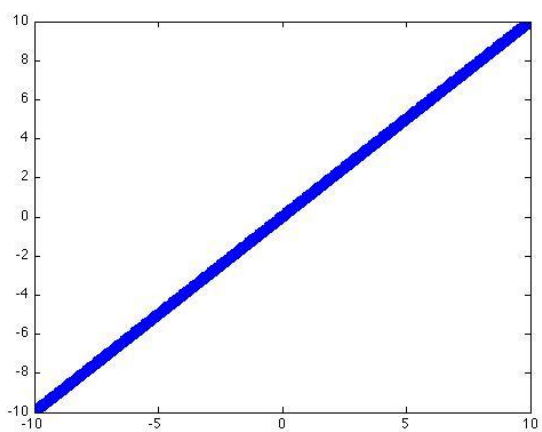
- The objective of logistic regression is to determine the probability that an observation falls into one of the two categories

Logistic Regression

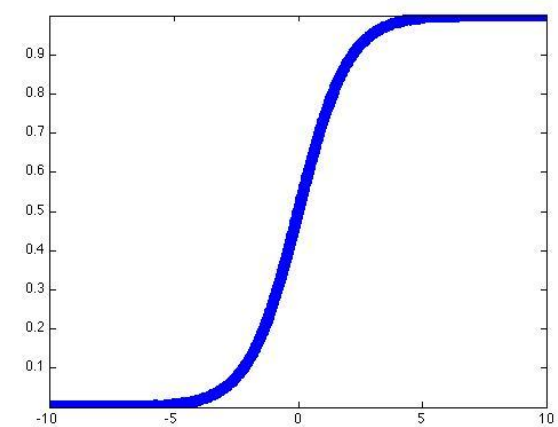
- *A form of Generalized Linear Model (GLM):*

$$y = g\left(\sum_{i=1}^{d} w_i x_i\right)$$

where: $$g(z) = \frac{1}{1+\exp(-z)}$$ (logit function)



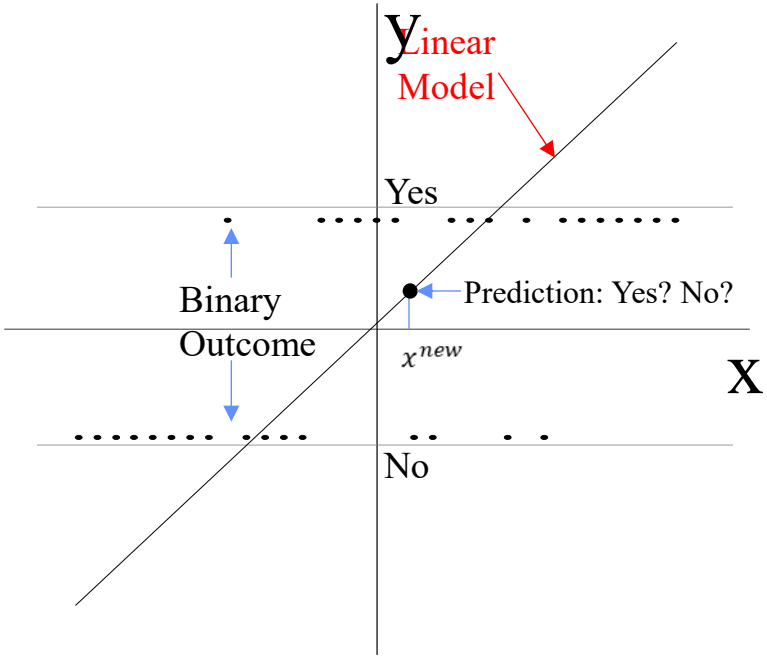
GLM regression

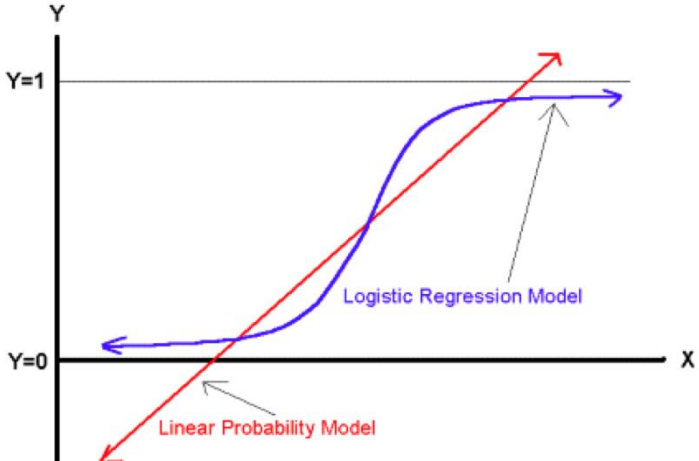


Logistic regression

Limitations of Linear Regression

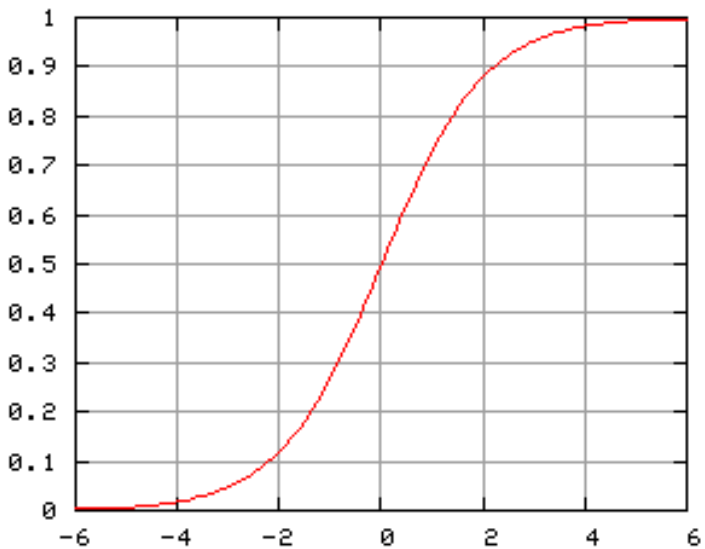
- *Fitting a linear model to data with binary outcome variable is problematic:*
  - This approach is analogous to fitting a linear model to the probability of the event, which can only take values between 0 (No) and 1 (Yes).
  - When given new x, the prediction is not restricted to be 0 or 1, which is difficult to interpret (i.e., whether Yes or No.)
- *Logistic regression model allows the dependent variable to be either 0 or 1.*
  - For this reason, logistic regression is widely used for BINARY classification problems.





Logistic Regression Model

- *Since they are probabilities, estimates of p will always be between 0 and 1.*
- *Logistic regression can be performed with one or more x variables that can be either continuous of binary.*
- *With only one x variable, the probability p of a focal event happening follows an s-shape with asymptotes at 0 and 1.*
- *For example, see the figure to the right.*



Logistic Regression Classification rule

$$\mathbf{P}\left(Y=1 \,|\, X\right) = \frac{1}{1+\exp\left(\sum_{i=1}^{d} w_i X_i\right)}$$

$$\mathbf{P}\left(Y=0 \,|\, X\right) = 1 - \frac{1}{1+\exp\left(\sum_{i=1}^{d} w_i X_i\right)}$$

$$= \frac{\exp\left(\sum_{i=1}^{d} w_i X_i\right)}{1+\exp\left(\sum_{i=1}^{d} w_i X_i\right)}$$

Logistic Regression Decision Surface

- *We say Y=0 when:* $1 < \dfrac{\mathbf{P}(Y=0 \mid X)}{\mathbf{P}(Y=1 \mid X)}$

- *Therefore:* $$\frac{\mathbf{P}(Y=0 \mid X)}{\mathbf{P}(Y=1 \mid X)} = \frac{\frac{\exp\left(\sum_{i=1}^{d} w_i X_i\right)}{1+\exp\left(\sum_{i=1}^{d} w_i X_i\right)}}{\frac{1}{1+\exp\left(\sum_{i=1}^{d} w_i X_i\right)}} = \exp\left(\sum_{i=1}^{d} w_i X_i\right)$$

- *Then, Y=0 when: :* $0 < \sum_{i=1}^{d} w_i X_i$

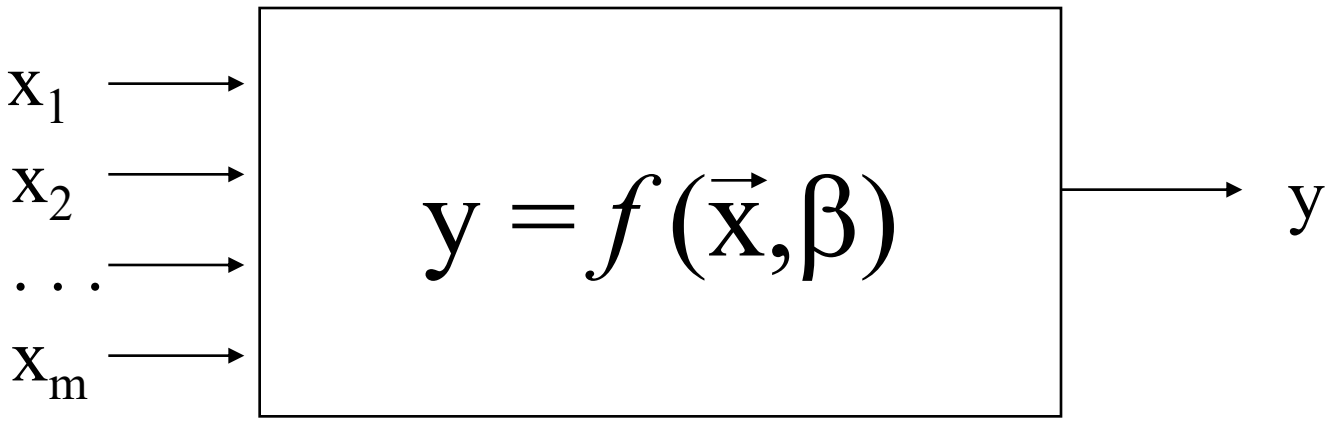
Fitting a 1-D Logistic Regression model

```
% Data
x = [-1 -0.5 0.3 0.5 0.8 0.9 1.2 1.7 2.3 2.5 2.8]';
y = [ 1 1 1 1 1 2 1 2 2 2 2]';

% Logistic regression fitting
[B,dev,stats] = mnrfit(x,y);

% Some model evaluations
pihat = mnrval(B,x)

xi = 1.3;  pihat = mnrval(B,xi)

I = (-2:0.1:3)';
pihat = mnrval(B,I)
p = exp(B(1)+B(2)*I) ./ (1 + exp(B(1)+B(2)*I));
plot(x,y,'*');hold on;
plot(I,pihat(:,2)+1);
hold off
legend('x', 'logistic regression')
```

Fitting a N-D Logistic Regression model in matlab

```
% Data
load fisheriris
x = meas;
y = double(categorical(species));

% Logistic regression fitting
[B,dev,stats] = mnrfit(x,y);

% Results
pihat = mnrval(B,x)

xi = [6.3, 2.8, 4.9, 1.7];
pihat = mnrval(B,xi)
```

Modelo NO Lineal genérico



Métodos iterativos

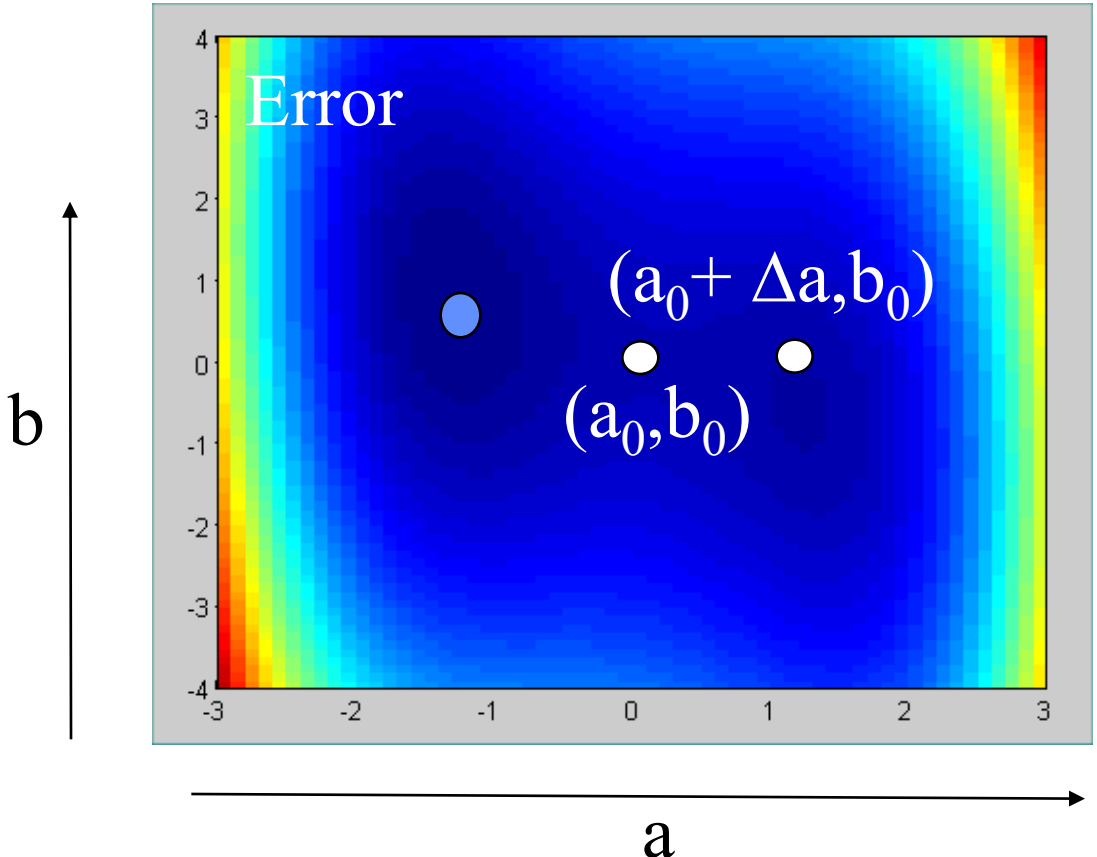
```
Definir una solución inicial (β0)
while(condición_de_fin)
      βi+1 ← g(βi)
end
```

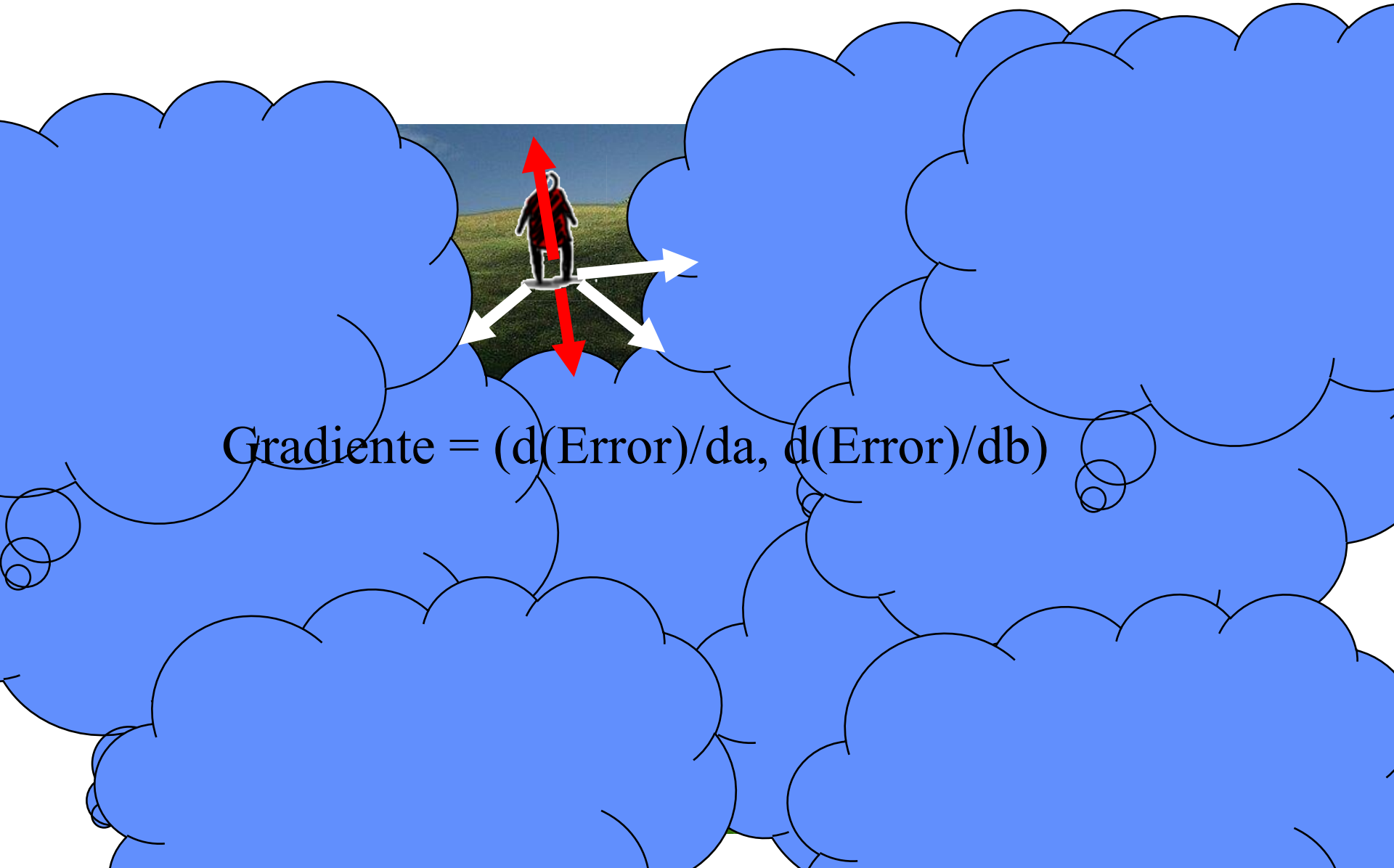
Ajuste de una recta

- $y = ax + b$

$dx = dE/da \approx (E(a + \Delta a) - E(a)) / \Delta a$

$dy = dE/db \approx (E(b + \Delta b) - E(b)) / \Delta b$





Gradiente = (d(Error)/da, d(Error)/db)

Mínimo de una función 2D

- *Supongamos que la superficie de error tiene una expresión conocida.*

$$E = x^4 - 3 \cdot x^2 + x + y^2 + x \cdot y$$

- *La solución analítica para los posibles mínimos vienen dados por:*

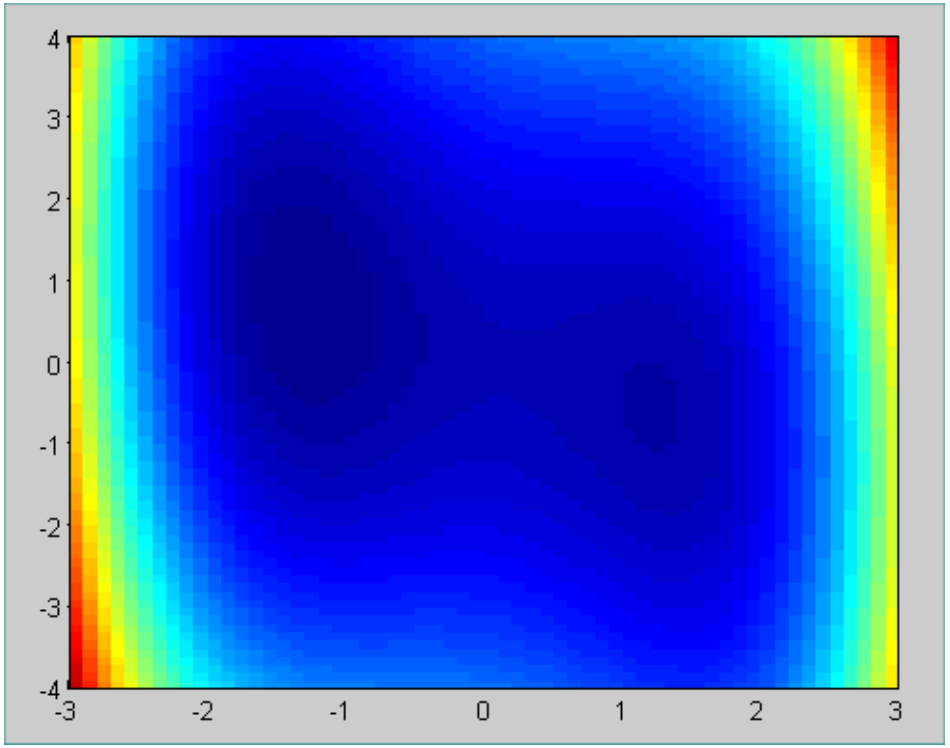
$$\frac{\partial E}{\partial x} = 4 \cdot x^3 - 6 \cdot x + 1 + y = 0 \qquad \frac{\partial E}{\partial y} = 2 \cdot y + x = 0$$

```
x = roots([4 0 -6.5 1]);
y = -x/2
```

x	y
-1.3457	0.6728
1.1895	-0.5947
0.1562	-0.0781

Mínimo de una función 2D

```
[x,y]=meshgrid(-3:0.1:3,-4:0.1:4);
z = (x.^4 - 3*x.^2 + x + y.*y + x.*y);
pcolor(x,y,z),shading flat
```



Posibles mínimos:

[-1.3457 , 0.6728]

[0.1562 , -0.0781]

[1.1895 , -0.5947]

Mínimo de una función 2D
Descenso del gradiente

- *El gradiente negativo siempre va en dirección hacia abajo. Sigámoslo.*
- *El algoritmo es el siguiente:*
  1. Elegir un punto inicial cualquiera ϕ_0
  2. Dirección = la del gradiente negativo

$$p_k = -\nabla f(\phi_k)$$

  3. Longitud del paso = α_k
  4. Modificar ϕ : $\phi_{k+1} = \phi_k + \alpha_k p_k$
  5. Volver al paso 2

Descenso del gradiente : Pincha en varios puntos iniciales y observa el comportamiento del algoritmo

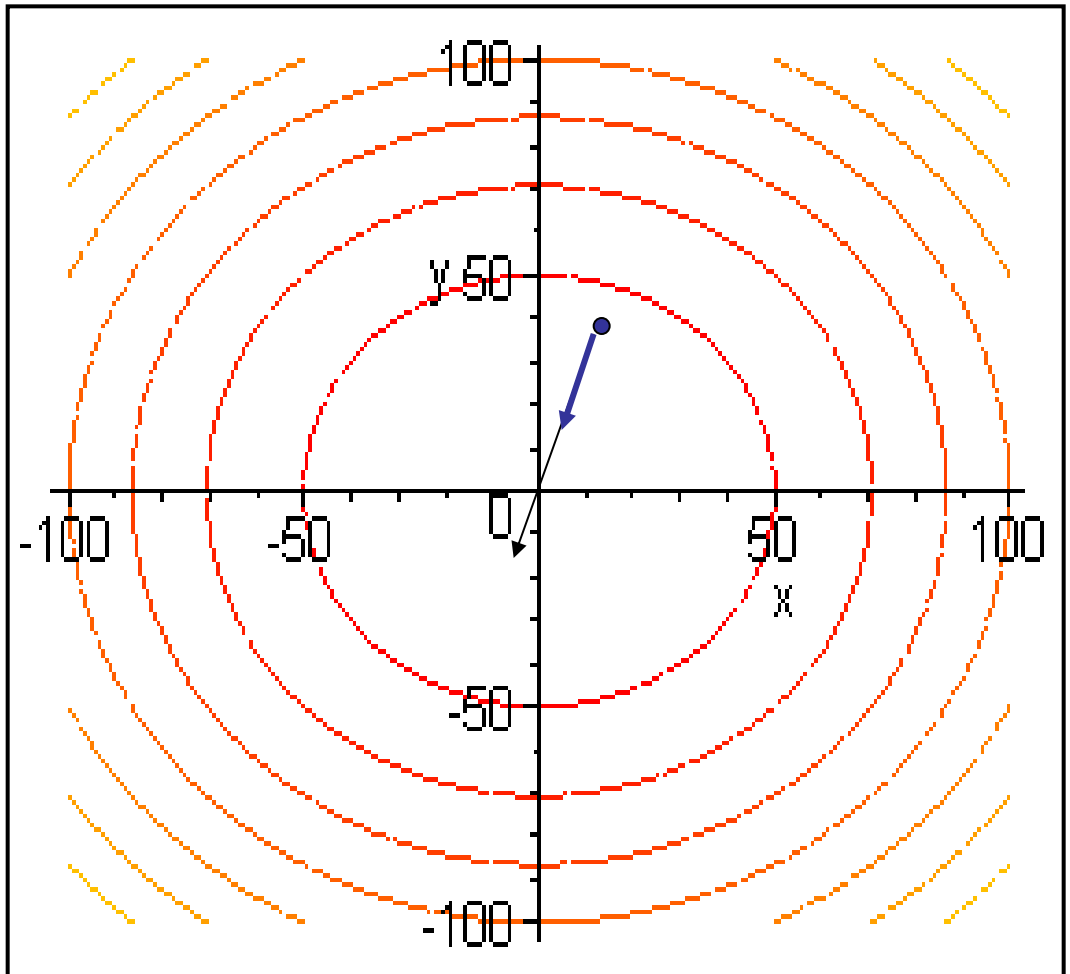
```
[x,y]=meshgrid(-3:0.1:3,-4:0.1:4);
z = (x.^4 - 3*x.^2 + x + y.*y + x.*y);
while(1),
    close all
    contour(x,y,z,100),shading flat;hold on;axis equal
    sol = ginput(1)';
    for i=1:100,
       x=sol(1,i);
       y=sol(2,i);
       dx = 4 * x.^3 - 6*x + 1 + y;
       dy = 2 * y + x;
       sol(:,i+1) = sol(:,i) - 0.01 * [dx;dy];
       plot([sol(1,i) sol(1,i+1)],[sol(2,i) sol(2,i+1)],'r', 'LineWidth',3)
       pause
    end;
end;
```

Descenso del gradiente : igual que antes, pero con el ejemplo de la recta

```
clear all
x = [1 2 4 4 5 6 7 8]';
y = [1 4 3 5 7 6 6 8]';
A = [x ones(size(x))];

weights = [0.8 1]';
P = weights;

m = -1:0.05:2;
b = -1:0.05:3;
[bgrid,mgrid] = meshgrid(b,m);
for i=1:length(m),
    for j=1:length(b),
        weights = [m(i) b(j)]';
        estim = A * weights;
        error(j,i) = log(sumsqr(y - estim));
    end
end

while(1),
   close all
   pcolor(m,b,error),shading interp,colorbar,hold on
   xlabel('b','FontSize',12)
   ylabel('m','FontSize',12)
   weights=ginput(1)';
   P=weights;
   for i=1:1000,
      estim = A * weights;
      aux = repmat(estim-y,1,size(A,2));
      gradiente = sum(aux.* A);
      weights = weights - 0.001 * gradiente';
      P = [P weights];
   end;
   comet(P(1,:),P(2,:));
   pause
end;
```

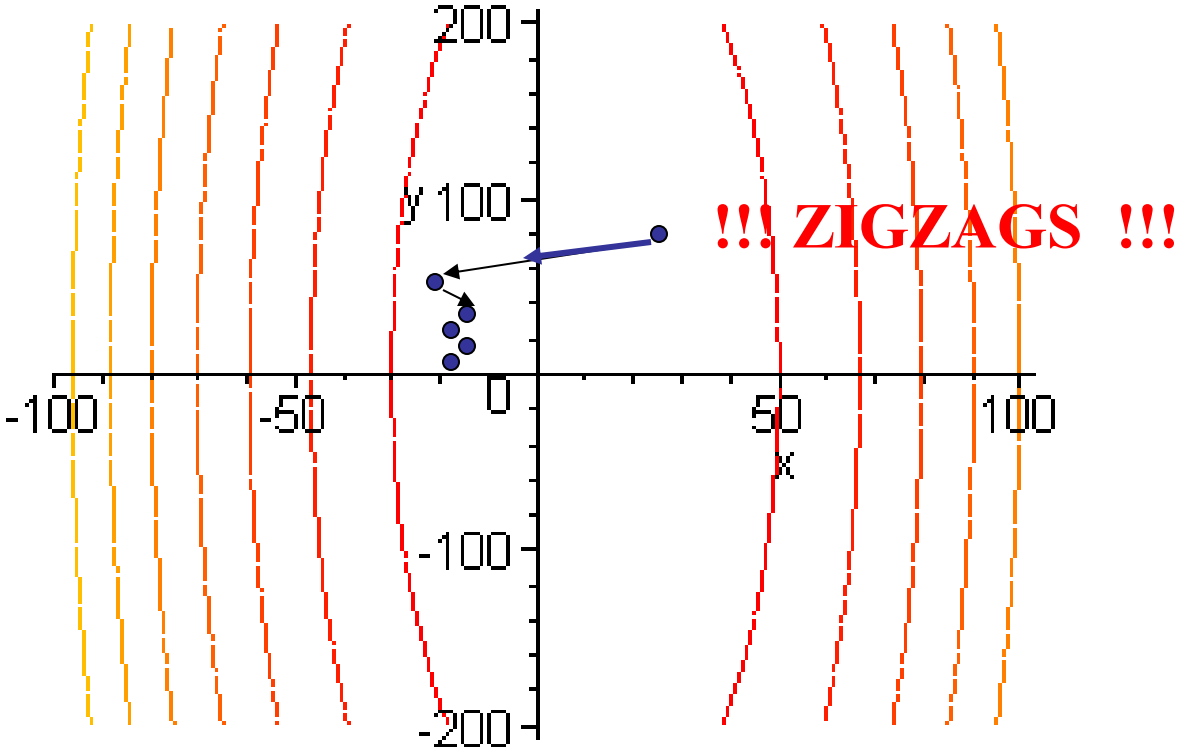
Problema bien condicionado

E = x^2+y^2



Problema mal condicionado

50(x-10)^2+y^2



Método de Newton

How to extend Newton's method to multivariable case?

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \frac{y'(\mathbf{x}_k)}{y''(\mathbf{x}_k)}$$ Is this correct?

Start again with Taylor expansion:

$$y(\mathbf{x}) = y(\mathbf{x}_k) + \nabla y(\mathbf{x}_k)(\mathbf{x}-\mathbf{x}_k) + 0.5\,(\mathbf{x}-\mathbf{x}_k)\mathbf{H}(\mathbf{x}_k)\,(\mathbf{x}-\mathbf{x}_k)$$

Note that $\mathbf{H}$ is the Hessian containing the second order derivatives.

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \frac{\nabla y(\mathbf{x}_k)}{\mathbf{H}(\mathbf{x}_k)}$$ Is this correct?

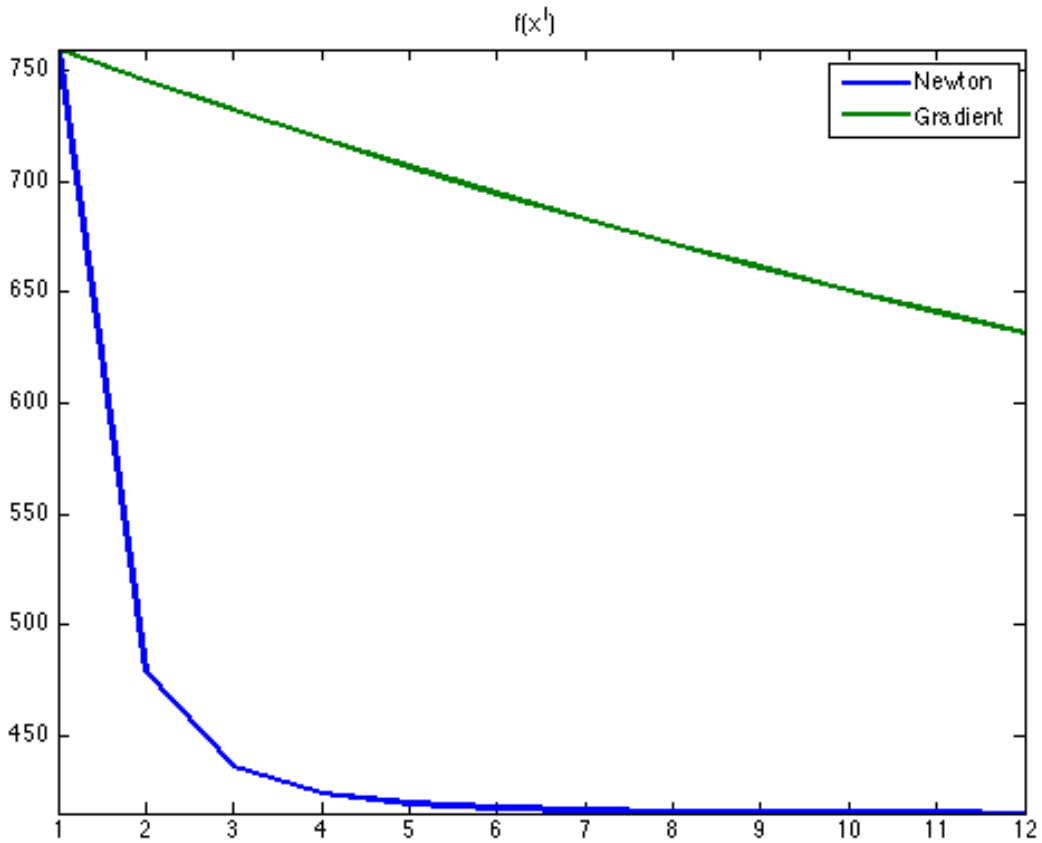
Newton's method for finding an extreme point is

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \mathbf{H}^{-1}(\mathbf{x}_k)\,\nabla y(\mathbf{x}_k)$$

Método de Newton : Pincha en varios puntos iniciales y observa el comportamiento del algoritmo

```
[x,y]=meshgrid(-3:0.1:3,-4:0.1:4);
z = (x.^4 - 3*x.^2 + x + y.*y + x.*y);

while(1),
    close all
    contour(x,y,z,100),shading flat;hold on;axis equal
    sol = ginput(1)';
    for i=1:100,
       x=sol(1,i);
       y=sol(2,i);
       dx = 4 * x.^3 - 6*x + 1 + y;
       dy = 2 * y + x;
       dxx = 12 * x.^2 - 6;         dxy = 1;
       dyx = 1;                     dyy = 2;
       H = [dxx dxy;dyx dyy];
       sol(:,i+1) = sol(:,i) - 0.1 * inv(H)  * ([dx;dy]);
       plot([sol(1,i) sol(1,i+1)],[sol(2,i) sol(2,i+1)],'r')
       pause
    end;
end;
```

Descenso gradiente vs. Newton



Algunos algoritmos de regresión no lineal

https://medium.com/@mohamadhasan.sarvandani/top-algorithms-of-non-linear-regression-in-machine-learning-proposed-python-library-and-python-c871752ddd81

Regularización

El problema del sobreajuste (overfitting)

- *Si el modelo es muy complejo respecto a la naturaleza de los datos, se produce el problema del overfitting*
- *Normalmente, esto se refleja en que algunos de los coeficientes obtenidos son anormalmente altos*

El problema del sobreajuste (overfitting)

```
rand('seed',0);
x = rand(1,10)-0.1;
y = 3*x.*x - 2*x + 3 + 0.1*randn(1,10);
plot(x,y,'*r'); a=axis; pause;

xi=linspace(-0.1,0.9,100);

for i=1:10
  p=polyfit(x,y,i);
  plot(x,y,'*r');hold on;
  plot(xi,polyval(p,xi),'b');hold off
  axis(a);
  title(p);pause
end
```

Regularización

- La regularización modifica la función a optimizar, añadiendo términos que dependen de los coeficientes
- Limita/restringe su valor para que sean cercanos a 0
- Obviamente, si los coeficientes del modelo lineal son pequeños, la salida del modelo también lo será
- Esto evita el problema del overfitting

- **WARNING: You will need to scale your data before using these regularization**

Métodos principales de regularización

- *OLS:* $argmin_{\beta} \sum_i (y_i - \boldsymbol{\beta}' \boldsymbol{x}_i)^2$

- *Lasso:* $argmin_{\beta} \sum_i (y_i - \boldsymbol{\beta}' \boldsymbol{x}_i)^2 + \lambda \sum_{k=1}^{K} |\beta_k|$

- *Ridge:* $argmin_{\beta} \sum_i (y_i - \boldsymbol{\beta}' \boldsymbol{x}_i)^2 + \lambda \sum_{k=1}^{K} \beta_k^2$

- *ElasticNet:*

$$argmin_{\beta} \sum_i (y_i - \boldsymbol{\beta}' \boldsymbol{x}_i)^2 + \lambda_1 \sum_{k=1}^{K} |\beta_k| + \lambda_2 \sum_{k=1}^{K} \beta_k^2$$

Solución técnicas regularización

1️⃣ Ridge Regression

Tiene **solución analítica cerrada**.

La solución de los coeficientes es:

$$\hat{\beta} = (X^T X + \lambda I)^{-1} X^T y$$

Método	¿Solución analítica?	¿Método de resolución habitual?
Ridge	✅ Sí	Fórmula cerrada
Lasso	❌ No	Métodos iterativos
Elastic Net	❌ No	Métodos iterativos

Least Absolute Shrinkage and Selection Operator (LASSO)

- El modelo LASSO penaliza los coeficientes con valores muy elevados añadiendo una penalización basada en la norma l^1:

$$argmin_{\beta} \sum_{i} (y_i - \boldsymbol{\beta}' \boldsymbol{x}_i)^2 + \lambda \sum_{k=1}^{K} |\beta_k|$$

- La regularizacion L1 hace que muchos pesos tiendan a cero, lo cual permite desechar la correspondiente variable de entrada
- Esto permite encontrar modelos donde muchas variables no se usan → SELECCIÓN DE VARIABLES !!!!
- *Alpha* mide en qué medida se penaliza la magnitude de los pesos

Ridge Regression

- *Ridge regression penalize the size of the regression coefficients based on their l^2 norm:*

$$argmin_{\beta} \sum_{i} (y_i - \boldsymbol{\beta}' \boldsymbol{x}_i)^2 + \lambda \sum_{k=1}^{K} \beta_k^2$$

- *The tuning parameter serves λ to control the relative impact of these two terms on the regression coefficient estimates.*
  - Selecting a good value for λ is critical; cross-validation is used for this.

Ridge regression

- **Ridge regression** takes a step further and penalizes the model for the sum of squared value of the weights
- L2 regularization cares more about driving big weight to small weights, instead of driving small weights to zeros.
- Thus, the weights not only tend to have smaller absolute values, but also really tend to penalize the extremes of the weights
- This results in a group of weights that are more evenly distributed

ElasticNet regression

- **ElasticNet** is a hybrid of Lasso and Ridge regression
- It minimizes

$$argmin_{\beta} \sum_i (y_i - \boldsymbol{\beta}' \boldsymbol{x}_i)^2 + \lambda_1 \sum_{k=1}^{K} |\beta_k| + \lambda_2 \sum_{k=1}^{K} \beta_k^2$$

- The values of λ_1 and λ_2 define the Lasso vs. Ridge proportion

```
N = 200;
weights = [0;2;0;-3;0];    % Only two nonzero coefficients
X = randn(N,5);
y = X*weights + randn(N,1)*0.01; % Small added noise

% Normalization (zero mean, unit variance)
m=mean(X);  s=std(X);
X=(X-repmat(m,N,1))./repmat(s,N,1);

% OLS
coefsOLS = pinv(X)*y;

% Ridge regression (el primer coef. devuelto es el termino independiente)
rp=0.1;         % Ridge parameter (0<rp<1)
coefsRIDGE = ridge(y,X,rp);

% LASSO
Lambda = 0.1;  % Regularization value
coefsLASSO = lasso(X,y,'Lambda',Lambda);

% Elastic Net
Alpha = 0.9;  % Elastic net value
coefsElasticNet = lasso(X,y,'Lambda',Lambda,'Alpha',Alpha);

coefs = [coefsOLS coefsRIDGE coefsLASSO coefsElasticNet]
```

Matlab parameters

- *ridge*
  - The first parameter is y, and the second one is X!!!
  - glmfit(X,y) es igual a ridge(y,X,0,0)
  - The fourth parameter activates centering and rescaling
- *lasso*
  - glmfit(X,y) es igual a lasso(X,y, 'Lambda',0)
  - alpha is the weight of lasso (L^1) versus ridge (L^2) optimization
    - alpha = 1 represents pure lasso regression
    - alpha close to 0 approaches ridge regression
  - By default, alpha=1

References

[1] Tibshirani, R. "Regression shrinkage and selection via the lasso." Journal of the Royal Statistical Society, Series B, Vol 58, No. 1, pp. 267–288, 1996.

[2] Zou, H. and T. Hastie. "Regularization and variable selection via the elastic net." Journal of the Royal Statistical Society, Series B, Vol. 67, No. 2, pp. 301–320, 2005.

[3] Friedman, J., R. Tibshirani, and T. Hastie. "Regularization paths for generalized linear models via coordinate descent." Journal of Statistical Software, Vol 33, No. 1, 2010. https://www.jstatsoft.org/v33/i01

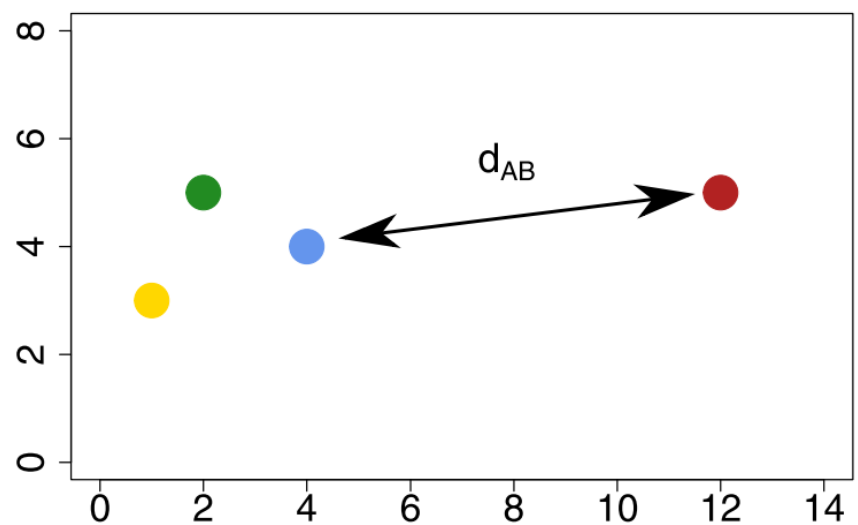
[4] Hastie, T., R. Tibshirani, and J. Friedman. The Elements of Statistical Learning, 2nd edition. Springer, New York, 2008.

Otras técnicas

Classical Multidimensional Scaling (CMS o MDS)

- MDS is used to translate the matrix of pairwise distances among a set of N objects into a map of N points in a Cartesian space **of desired dimension**

	A	B	C	D
A	0	d_{AB}	d_{AC}	d_{AD}
B	d_{AB}	0	d_{BC}	d_{BD}
C	d_{AC}	d_{BC}	0	d_{CD}
D	d_{AD}	d_{BD}	d_{CD}	0



Classical Multidimensional Scaling (CMS o MDS)

```
% Reconstructing Spatial Locations from Nonspatial Distances
X = [39.1 40.7 41.5 39.2 38.7 41.7 40.1 39.2
     18.7 21.2 21.5 21.8 20.6 20.1 22.1 21.6]';

% Distance between points
distances = pdist(X);

% Classical Multidimensional Scaling: how to recover X from distances
[Y,eigvals] = cmdscale(distances);

% Procrustes analysis: it determines a linear transformation (translation,
% reflection, orthogonal rotation, and scaling) of the points in the
% matrix Y to best conform them to the points in the matrix X
[D,Z,transform] = procrustes(X,Y(:,1:2));

% Plot original data vs. reconstruction
plot(X(:,1),X(:,2),'ro',Z(:,1),Z(:,2),'bx');
```

MDS on IRIS database

```
load iris; y = y+1;
distM = pdist(x');

% MDS : obtiene un mapa de N dimensiones a
partir de las distancias. Si dim=4, los datos
deben ser los originales
% newx = cmdscale(distM,4);

newx = cmdscale(distM,2); % Reducir a dim=2
plotpat(newx',y)
```

MDS en ciudades españolas

https://es.mejoresrutas.com/tabla-de-distancias-entre-ciudades/es/

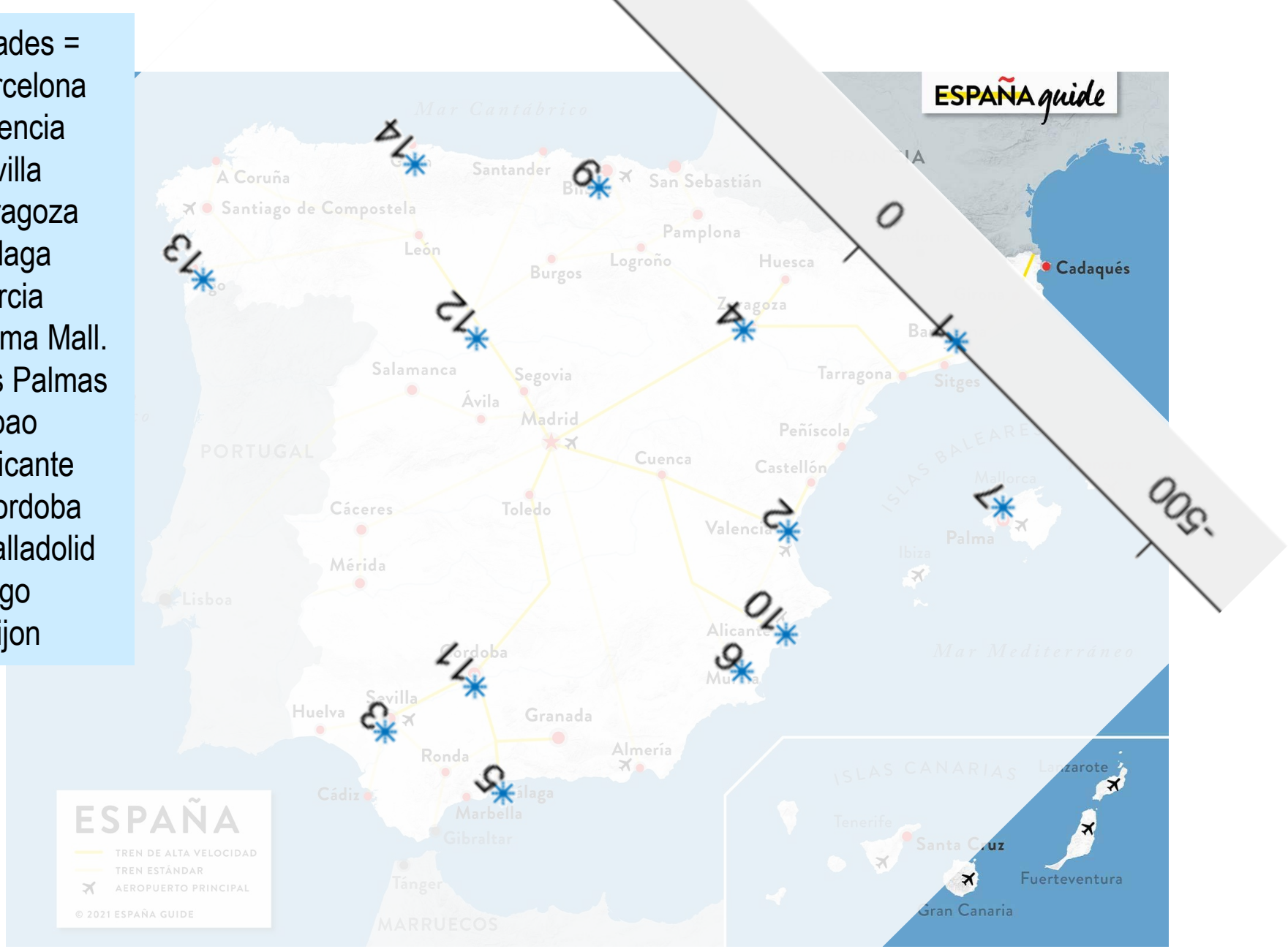
```
M = [...
0 303 831 256 770 471 206 2175 469 407 711 576 908 686
303 0 541 246 468 177 260 1874 473 125 421 441 766 632
831 541 0 646 158 433 791 1355 703 495 121 486 588 685
256 246 646 0 628 408 377 2001 246 368 535 320 652 444
770 468 158 628 0 323 695 1406 739 391 133 549 716 766
471 177 433 408 323 0 372 1723 606 69 320 510 800 726
206 260 791 377 695 372 0 2093 621 303 671 664 1000 820
2175 1874 1355 2001 1406 1723 2093 0 2022 1792 1470 1790 1682 1925
469 473 703 246 739 606 621 2022 0 583 618 232 486 223
407 125 495 368 391 69 303 1792 583 0 379 515 821 722
711 421 121 535 133 320 671 1470 618 379 0 419 589 634
576 441 486 320 549 510 664 1790 232 515 419 0 337 224
908 766 588 652 716 800 1000 1682 486 821 589 337 0 289
686 632 685 444 766 726 820 1925 223 722 634 224 289 0];
```

Ciudades =
1 Barcelona
2 Valencia
3 Sevilla
4 Zaragoza
5 Malaga
6 Murcia
7 Palma Mall.
8 Las Palmas
9 Bilbao
10 Alicante
11 Cordoba
12 Valladolid
13 Vigo
14 Gijon

MDS en ciudades españolas

```
% Convierte la matriz de distancias en un vector fila
de distancias (sin datos repetidos ni distancias nulas)
A = tril(M,0);
A = A(find(A))';

% MDS a 2 dimensiones
newx = cmdscale(A,2);

% Plot
plot(newx(:,2),newx(:,1),'*');hold on;
for i=1:size(newx,1)
  text(newx(i,2)+10,newx(i,1),num2str(i));
end
axis equal
```

Ciudades =
1 Barcelona
2 Valencia
3 Sevilla
4 Zaragoza
5 Malaga
6 Murcia
7 Palma Mall.
8 Las Palmas
9 Bilbao
10 Alicante
11 Cordoba
12 Valladolid
13 Vigo
14 Gijon



ANEXO 1

Derivación matemática OLS(1 entrada)

Modelo lineal
Ajuste a una recta (m=1)

- *Consideraremos inicialmente una sola x*
- *Es decir, a partir del conjunto de datos*

$$\{(x_i, y_i) \quad i = 1,\ldots, n\}$$

buscamos la función

$$\hat{y} = ax + b$$

que mejor aproxime a estos datos.

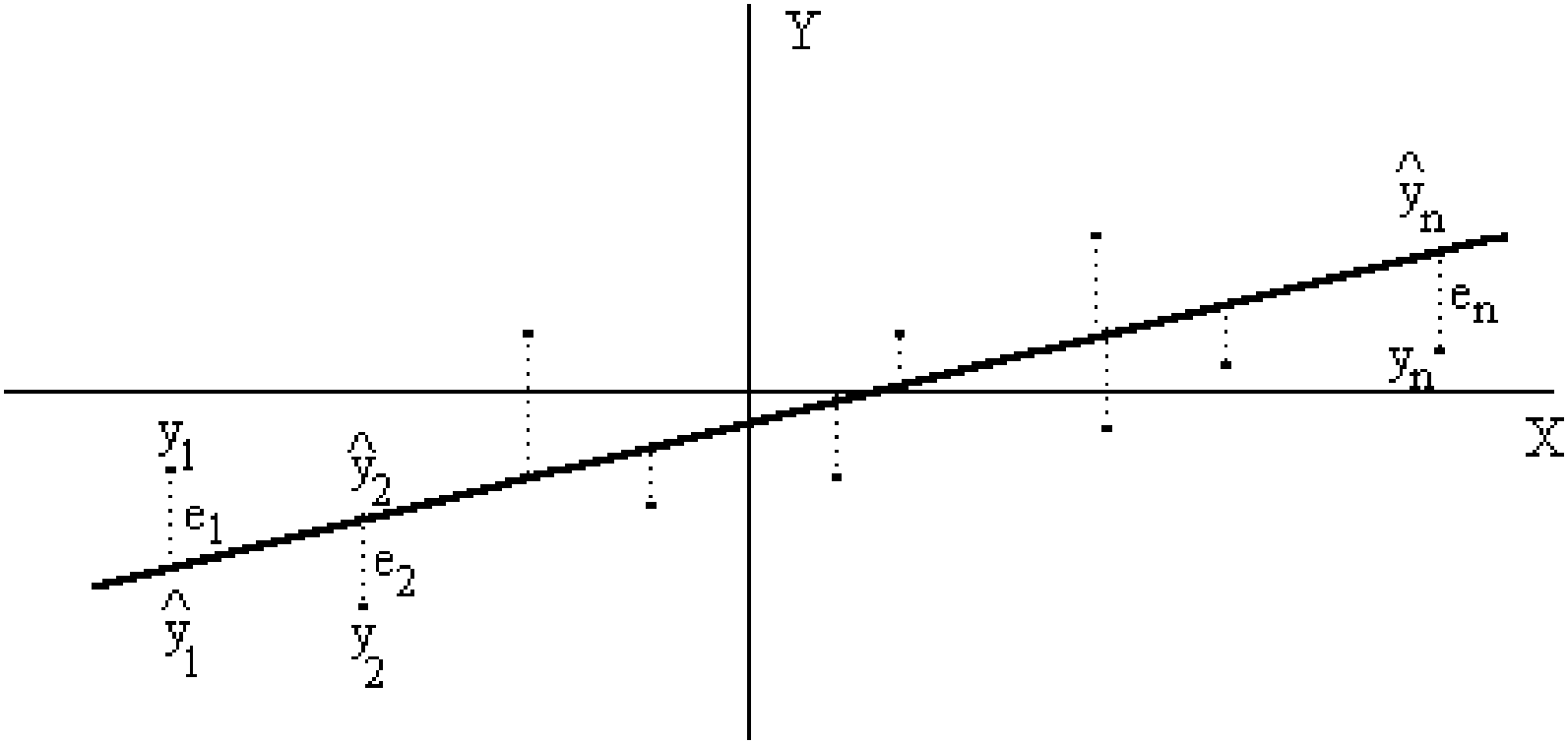
- *O sea, cuánto valen ¿a? ¿b?*

Modelo lineal

Ajuste a una recta (m=1)

1. *Definir el error en cada punto*

$$e_i = y_i - \hat{y}(x_i) = y_i - (ax_i + b)$$



Modelo lineal

Ajuste a una recta (m=1)

2. *Definir un criterio de error*

$$E(a,b) = \sum_{i=i}^{n} e_i^{\,2} = \sum_{i=i}^{n} (y_i - (ax_i + b))^2$$

- *Otros criterios podrían ser, por ejemplo, la suma de los valores absolutos de los errores*
- *El motivo de utilizar las suma de los errores al cuadrado es la posibilidad de derivar el error con facilidad, como veremos a continuación*

Modelo lineal

Ajuste a una recta (m=1)

3. Derivar respecto de cada parámetro e igualar a 0

$$\left.\begin{aligned}\frac{\partial E(a,b)}{\partial a}&=0\\ \frac{\partial E(a,b)}{\partial b}&=0\end{aligned}\right\}\qquad \begin{aligned}\frac{\partial E}{\partial a}&=2\sum_{i=1}^{n}\left(y_i-(ax_i+b)\right)(-x_i)\\ \frac{\partial E}{\partial b}&=2\sum_{i=1}^{n}\left(y_i-(ax_i+b)\right)(-1)\end{aligned}$$

E igualando a 0, obtenemos:

$$\begin{pmatrix}\sum_{i=1}^{n}x_i^{\,2} & \sum_{i=1}^{n}x_i\\ \sum_{i=1}^{n}x_i & n\end{pmatrix}\begin{pmatrix}a\\ b\end{pmatrix}=\begin{pmatrix}\sum_{i=1}^{n}y_ix_i\\ \sum_{i=1}^{n}y_i\end{pmatrix}$$

ANEXO 2

Derivación matemática GLM (varias entradas)

Modelo lineal generalizado (GLM) (m>1)

- *Sea* $\{(\vec{x}_i, y_i) \quad i = 1,\ldots, n\}$ $\qquad n = n^o$ *de datos*

  donde $\vec{x}_i = (x_{i1}, x_{i2}, \ldots, x_{im})$ $\qquad m = n^o$ *caract.*

- *En este caso:*

$$\hat{y} = a_1 x_1 + a_2 x_2 + \ldots + a_m x_m + a_0$$

Modelo lineal generalizado (GLM)

1. *Definir el error en cada punto*

$$e_i = y_i - \hat{y}(\vec{x}_i) = y_i - \left(a_1 x_{i1} + a_2 x_{i2} + \ldots + a_m x_{im} + a_0\right)$$

$$\underset{r}{\begin{pmatrix} e_1 \\ e_2 \\ \vdots \\ e_n \end{pmatrix}} = \underset{b}{\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}} - \underbrace{\begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1m} & 1 \\ x_{21} & x_{22} & \cdots & \vdots & 1 \\ \vdots & \vdots & \ddots & \vdots & 1 \\ x_{n1} & x_{n2} & \cdots & x_{nm} & 1 \end{pmatrix}}_{A} \cdot \underset{x}{\begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_0 \end{pmatrix}}$$

$$r_{nx1} = b_{nx1} - A_{nx(m+1)} x_{(m+1)x1}$$

Modelo lineal generalizado (GLM)

2. *El criterio de error será:*

$$E = \sum_{i=i}^{n} e_i^{\,2} = \begin{pmatrix} e_1 & e_2 & \cdots & e_n \end{pmatrix} \cdot \begin{pmatrix} e_1 \\ e_2 \\ \vdots \\ e_n \end{pmatrix} = r^t \cdot r$$

$$E = r^t \cdot r = (b - Ax)^t (b - Ax) =$$

$$b^t b - b^t (Ax) - (Ax)^t b + (Ax)^t (Ax) =$$

$$b^t b - b^t Ax - x^t A^t b + x^t A^t Ax$$

Dado que $b^t Ax$ *es de dimensiones 1 x 1, se cumple:*

$$b^t Ax = \left(b^t Ax\right)^t = x^t A^t b$$

Por lo que $E = b^t b - 2x^t A^t b + x^t A^t Ax$

Modelo lineal generalizado (GLM)

- *Para poder derivar E respecto de x, es preciso saber cómo se deriva respecto de un vector*
- *De forma general, sea z un vector columna, y M una matriz, se cumple que:*

$$\frac{\partial z^t M}{\partial z} = M \qquad \frac{\partial z^t M z}{\partial z} = \left(M + M^t\right) \cdot z$$

$$\frac{\partial z^t M^t b}{\partial z} = M^t b$$

Modelo lineal generalizado (GLM)

3. *Derivando el error respecto del vector x de parámetros:*

$$\frac{\partial E}{\partial x} = \frac{\partial\left[b^t b - 2x^t A^t b + x^t A^t A x\right]}{\partial x} = \frac{\partial\left[b^t b\right]}{\partial x} - \frac{\partial\left[2x^t A^t b\right]}{\partial x} + \frac{\partial\left[x^t A^t A x\right]}{\partial x}$$

$$\left.\begin{aligned} &\frac{\partial\left[b^t b\right]}{\partial x} = 0 \\ &\frac{\partial\left[2x^t A^t b\right]}{\partial x} = 2A^t b \\ &\frac{\partial\left[x^t A^t A x\right]}{\partial x} = \left(A^t A + \left(A^t A\right)^t\right)x = 2A^t A x \end{aligned}\right\} \Rightarrow \quad \frac{\partial E}{\partial x} = 0 - 2A^t b + 2A^t A x$$

Modelo lineal generalizado (GLM)

3. *... e igualando a 0, se obtiene:*

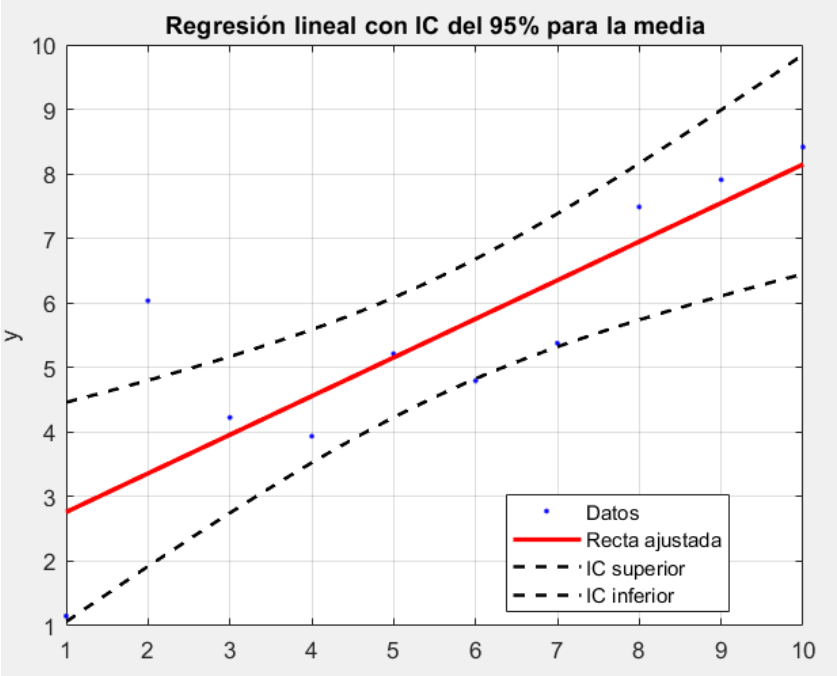
$$-2A^t b + 2A^t Ax = 0$$

$$A^t b = A^t Ax$$

$$\boxed{x = \underbrace{\left(A^t \cdot A\right)^{-1} \cdot A^t}_{\text{Pseudoinversa de A}} \cdot b}$$

ANEXO 3

Intervalos de confianza en un modelo de regresión lineal



```
x = (1:10)';    y = 2 + 0.5*x + randn(size(x)); % datos con ruido
n = length(x);

% Ajuste lineal
p = polyfit(x,y,1);
y_hat = polyval(p,x);

% Cálculo de parámetros necesarios
x_mean = mean(x);
Sxx    = sum((x - x_mean).^2);
SSE    = sum((y - y_hat).^2);
s      = sqrt(SSE/(n-2)); % error estándar residual

% Mallado fino para dibujar curvas suaves
x0 = linspace(min(x), max(x), 200)';
y0 = polyval(p,x0);

% Nivel de confianza 95%
alpha = 1-0.95;
tval = tinv(1 - alpha/2, n-2);

% Intervalo de confianza de la MEDIA
IC = tval * s * sqrt(1/n + (x0 - x_mean).^2 / Sxx);

figure;
plot(x,y,'b.'); hold on
```

$$\hat{y}(x_0) \pm t_{\alpha/2,n-2}\, s \sqrt{\frac{1}{n} + \frac{(x_0 - \bar{x})^2}{\sum (x_i - \bar{x})^2}}$$